5 CHAPTER 5: SYSTEM ANALYSIS AND DESIGN

This chapter presents the system analysis and design of the proposed USIU Cafeteria Ordering System. It outlines the structural and functional models that define how the system operates, interacts with users, and achieves its intended objectives. The chapter introduces a series of diagrams - system architecture, use case, entity-relationship, data flow, flowcharts, activity, sequence, and class diagrams - which collectively illustrate the internal workings of the system. Wireframes are provided to showcase the user interface design across all five main screens. By integrating both conceptual and visual representations, this chapter establishes a clear foundation for the implementation phase, ensuring that the ordering, wallet, and pre-order features are both technically sound and user-friendly.

5.1 System Models

This section presents the system models developed for the USIU Cafeteria Ordering System. Each model provides a different perspective on how the system is structured, how data flows through it, and how users interact with it. Together the models form a comprehensive blueprint that guides the implementation phase and ensures every feature - ordering, wallet management, pre-order scheduling, and staff operations - is logically consistent and technically feasible.

5.1.1 System Architecture Diagram

A system architecture diagram is a visual representation that shows how the different components of a system interact and work together. It illustrates the structure, data flow, and relationships between the user interface, application logic, and data storage layers. For the USIU Cafeteria Ordering System, the diagram outlines how students, staff, and administrators - across two client applications - connect to Firebase backend services and Supabase Edge Functions to deliver real-time ordering, wallet management, and pre-order scheduling.

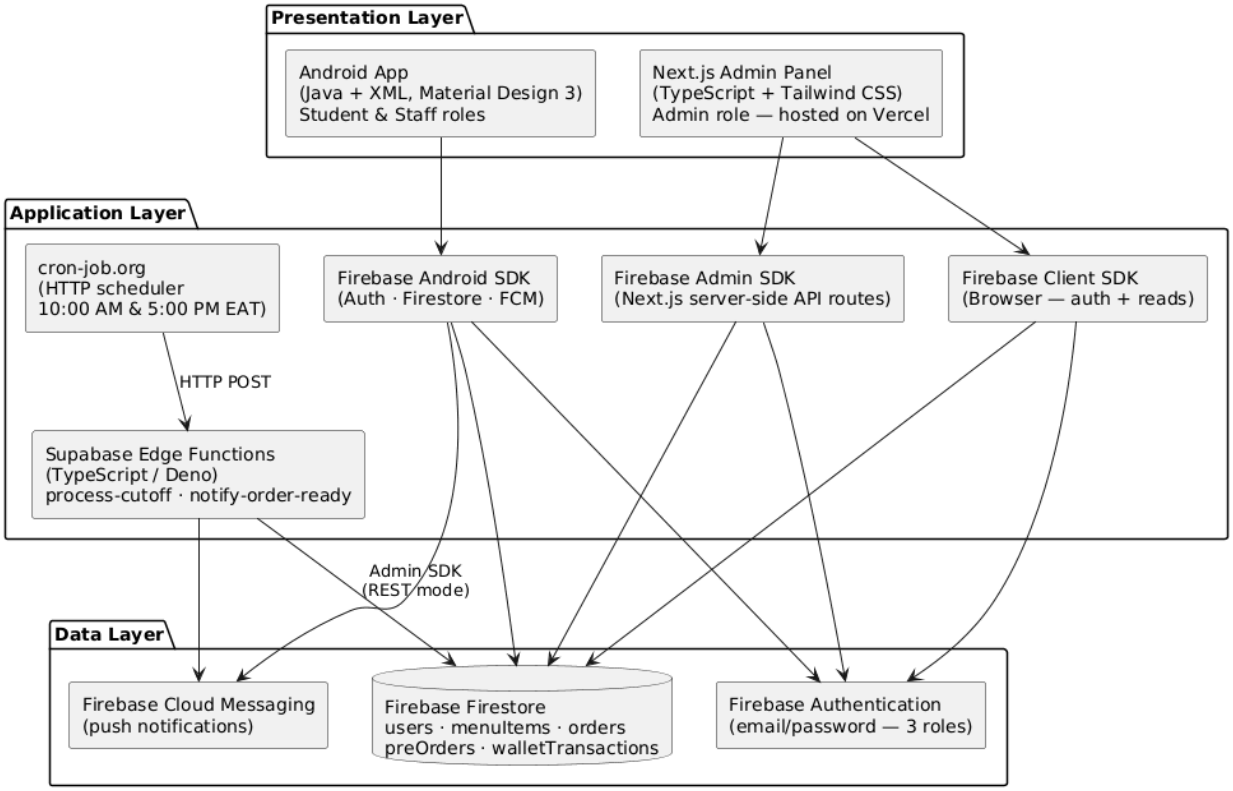


Figure 12: System Architecture Diagram

Figure 12 above shows that the architecture of the USIU Cafeteria Ordering System follows a three-tier design consisting of the Presentation Layer, Application Layer, and Data Layer.

1. Presentation Layer

This layer represents the user interface and the primary interaction point of the system. It consists of two separate client applications serving three user roles: Student, Staff (Cashier), and Admin.

The *native Android application* (Java + XML, Material Design 3) serves both Students and Staff. Students access five main screens: Menu (browse and filter by category), Cart (build order, select wallet or cash payment), Orders (track active order, view history, see estimated wait time), Pre-orders (schedule and manage advance meals), and Profile/Wallet (balance, transaction history, account details). Staff access three operational screens: Staff Orders (view and update all incoming orders), Staff Menu (toggle item availability), and Staff Wallet (top up or deduct student wallet balances and record each transaction with the staff member's ID). Role-based navigation is determined at login by reading the role field from the user's Firestore document. Staff accounts

cannot self-register - they are created by an administrator and must change their temporary password on first login.

The *Next.js web admin panel* (TypeScript, Tailwind CSS, hosted on Vercel) serves Administrators only. It provides full menu management (add, edit, delete items; toggle availability; set image URLs), staff account creation (with a temporary password and an enforced first-login password change on the Android app), and a user directory with separate views for students and staff/admin accounts.

2. Application Layer

The Application Layer sits between the client applications and the Firebase backend. For the Android app, it is implemented through the Firebase Android SDK, which manages authentication sessions, Firestore real-time listeners, and FCM token registration on the device. For the admin panel, authentication and real-time reads use the Firebase Client SDK in the browser, while privileged operations such as creating staff accounts use the Firebase Admin SDK via Next.js server-side API routes.

On the server side, two *Supabase Edge Functions* (TypeScript/Deno runtime, deployed on the Supabase free plan) replace Firebase Cloud Functions for scheduled and event-driven logic. The process-cutoff function is invoked at 10:00 AM EAT (lunch) and 5:00 PM EAT (dinner) daily by cron-job.org. It queries all preOrders documents due that day, then for each: deducts the wallet balance via a Firestore transaction and marks the pre-order confirmed, or cancels it and sends an FCM notification if funds are insufficient. The notify-order-ready function is called directly by the Android app via an authenticated HTTP POST after a staff member marks an order as Ready, dispatching an FCM push notification to the student. All Firestore writes involving wallet deductions use runTransaction() to guarantee atomicity.

3. Data Layer

The Data Layer is managed by Firebase and consists of:

- *Firebase Firestore:* The primary cloud NoSQL database storing five collections - users, menuItems, orders, preOrders, and walletTransactions - with orderItems embedded

as arrays within each order and pre-order document. Real-time snapshot listeners push changes to the Android client without polling.

- *Firebase Authentication:* Manages identity for all three roles (student, staff, admin) through email/password sign-in. Each user has a corresponding document in the users collection with a role field.
- *Firebase Cloud Messaging:* Delivers push notifications to registered FCM tokens stored in the fcmToken field of each user's document.

Supabase Edge Functions connect to Firebase using the Firebase Admin SDK configured for REST transport (gRPC is not supported in the Deno edge runtime). cron-job.org provides external scheduling for the pre-order cut-off triggers, and Vercel provides continuous deployment for the admin panel - both at zero cost.

5.1.2 Use Case Diagram

A Use Case Diagram is used to model the interactions between users (actors) and the system, illustrating the different services the application provides. For the USIU Cafeteria Ordering System, the diagram identifies three human actors and one system actor, and the use cases each is associated with.

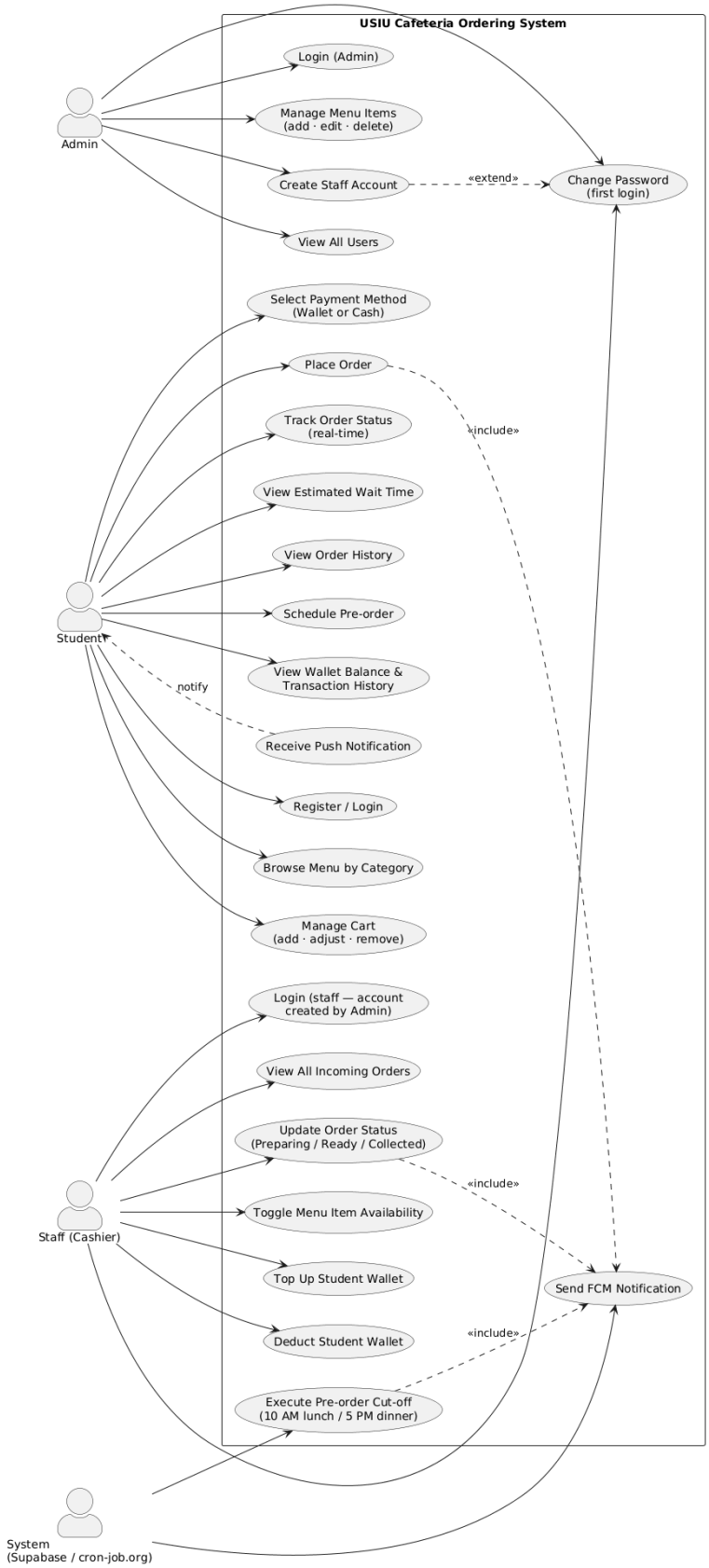


Figure 13: Use Case Diagram

Figure 13 above shows the use case diagram illustrating the key interactions between actors and the USIU Cafeteria Ordering System.

Student is the primary actor, interacting via the Android app. Use cases include: Register/Login, Browse Menu by Category, Add Item to Cart, Adjust Item Quantity, Select Payment Method (Wallet or Cash), Place Order, Track Order Status (real-time), View Estimated Wait Time, Receive Push Notification (Order Ready / Pre-order Confirmed / Insufficient Funds), View Order History, Schedule Pre-order, Set Recurring Pre-order, Cancel Pre-order (before cut-off), View Wallet Balance, and View Wallet Transaction History.

Staff (Cashier) is the second human actor, interacting via the Android app. Use cases include: Login (account created by Admin - staff cannot self-register), Change Password on First Login, View All Incoming Orders (real-time), Mark Order as Preparing, Mark Order as Ready, Mark Order as Collected, Toggle Menu Item Availability, Top Up Student Wallet (after receiving cash at counter), and Deduct Student Wallet Balance (manual charge adjustment, recorded with staff ID for audit).

Admin is the third human actor, interacting exclusively via the Next.js web admin panel. Use cases include: Login, Change Password on First Login, Create Staff Account (generates temporary password; forces first-login change on Android), Add Menu Item, Edit Menu Item, Delete Menu Item, Toggle Menu Item Availability, and View All Users (students and staff/admin separately).

System (Automated Functions) represents the Supabase Edge Functions and cron-job.org scheduler. Automated use cases include: Execute Lunch Pre-order Cut-off (10:00 AM EAT daily - triggered by cron-job.org POST to process-cutoff edge function), Execute Dinner Pre-order Cut-off (5:00 PM EAT - same logic), Send FCM Notification on Order Ready (Android calls notify-order-ready edge function after staff status update), Send FCM Notification on Pre-order Confirmed, and Send FCM Notification on Pre-order Cancelled due to Insufficient Funds.

The diagram uses «include» relationships for shared processes such as authentication (required by all login-gated use cases) and Firestore read/write (required by all data operations). «extend» relationships cover optional or conditional processes such as recurring pre-order chain creation, cancellation notifications, and forced first-login password change.

5.1.3 Entity Relationship Diagram (ERD)

Introduction

An ERD illustrates the logical structure of the database by showing entities, their attributes, and the relationships between them. For the USIU Cafeteria Ordering System, the ERD maps the five Firestore collections that store all system data - users, menuItems, orders, preOrders, and walletTransactions (with orderItems embedded as arrays) - and defines how they relate to one another.

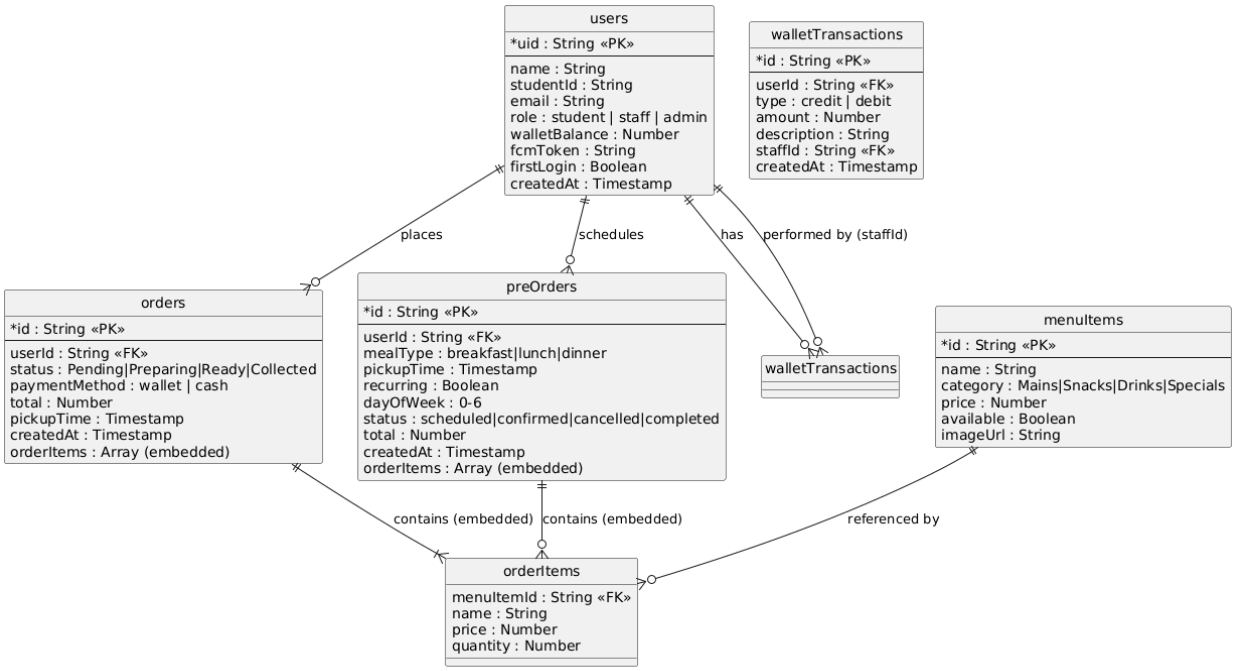


Figure 14: Entity Relationship Diagram (ERD)

Figure 14 above shows the Entity-Relationship Diagram for the USIU Cafeteria Ordering System. The five key entities are as follows:

users - Represents all registered users of the system. Attributes: uid (PK), name, studentId (students only; absent for staff and admin), email, role (student | staff | admin), walletBalance (students only; absent for staff and admin), fcmToken (renamed from deviceToken in the original plan), firstLogin (boolean; staff and admin only - set to true on account creation, cleared to false after the first-login password change), createdAt. A student user can place many orders, many pre-orders, and have many wallet transactions.

menuItems - Represents each item available in the cafeteria menu. Attributes: id (PK), name, category (Mains | Snacks | Drinks | Specials), price, available (boolean), imageUrl. A menu item can appear in many order items and many pre-order items.

orders - Represents a single order placed by a student. Attributes: id (PK), userId (FK → users), status (Pending | Preparing | Ready | Collected), paymentMethod (wallet | cash), total, pickupTime, createdAt. Each order belongs to one user and contains one or more embedded orderItems.

orderItems - Embedded within each order document as an array. Attributes: menuItemId (FK → menuItems), name, price, quantity. This denormalised structure avoids a separate collection lookup on every order read.

preOrders - Represents an advance meal scheduled by a student. Attributes: id (PK), userId (FK → users), mealType (breakfast | lunch | dinner), pickupTime, recurring (boolean), dayOfWeek (0–6, used if recurring), status (scheduled | confirmed | cancelled | completed), total, createdAt. Items are embedded as an array identical to orderItems.

walletTransactions - Represents each credit or debit event on a student's wallet. Attributes: id (PK), userId (FK → users - the student whose wallet was changed), type (credit | debit), amount, description, staffId (FK → users - the UID of the staff member who performed the transaction, for audit purposes), createdAt. Provides a complete, accountable audit trail for every wallet balance change, linking each transaction to the responsible staff member.

The ERD illustrates that all financial and ordering activity is anchored to the users entity through foreign key relationships. Firestore security rules use three helper functions - isStudent(), isStaff(), and isAdmin() - to gate access per collection, with an isPrivileged() helper combining staff and admin permissions for shared operations such as order status updates and menu management. This ensures each student sees only their own data while staff and admin can access the records necessary for their respective roles.

5.1.4 Data Flow Diagram (DFD)

A Data Flow Diagram (DFD) is a visual representation of how data moves within a system. It focuses on the flow of information between processes, data stores, and external entities, showing how inputs are transformed into outputs. The DFD for the USIU Cafeteria Ordering System is developed across three levels, from a high-level context view down to detailed sub-process breakdowns.

i. Level 0: Context Diagram

The Level 0 DFD represents the entire system as a single process and shows how it interacts with external entities at the highest level of abstraction.

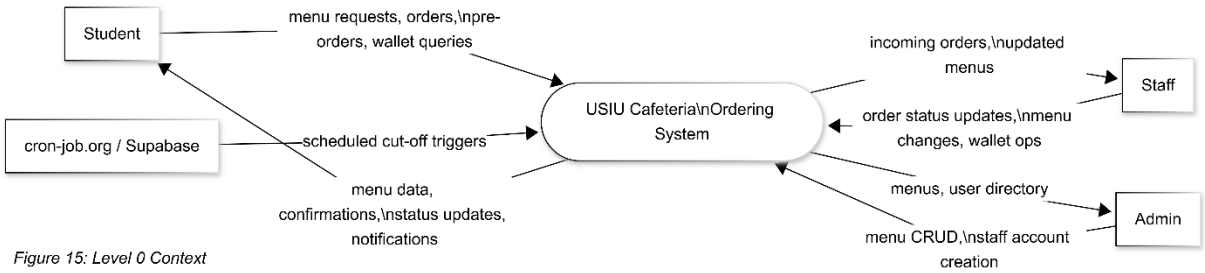


Figure 15: Level 0 Context Diagram

Figure 15 above shows the context diagram. The three external human entities are Student, Staff, and Admin. Students provide inputs - menu browsing requests, order data (items, quantity, payment method), pre-order schedules, and wallet queries. The system returns menu data, order confirmations, real-time status updates, estimated wait times, push notifications, and wallet balance information. Staff provide inputs - order status updates (Preparing/Ready/Collected), menu availability changes, and wallet top-up or deduction actions. The system returns incoming order lists and updated menu states to staff. Admin provides inputs via the web panel - menu item CRUD and staff account creation. The system returns updated menu lists and user directories to admin. The Supabase Edge Function (triggered by cron-job.org) interacts with the system as an automated entity at 10:00 AM and 5:00 PM EAT, providing scheduled cut-off processing that drives pre-order wallet deductions and cancellations.

ii. Level 1: Detailed DFD

The Level 1 DFD decomposes the single context process into the main functional sub-processes of the system.

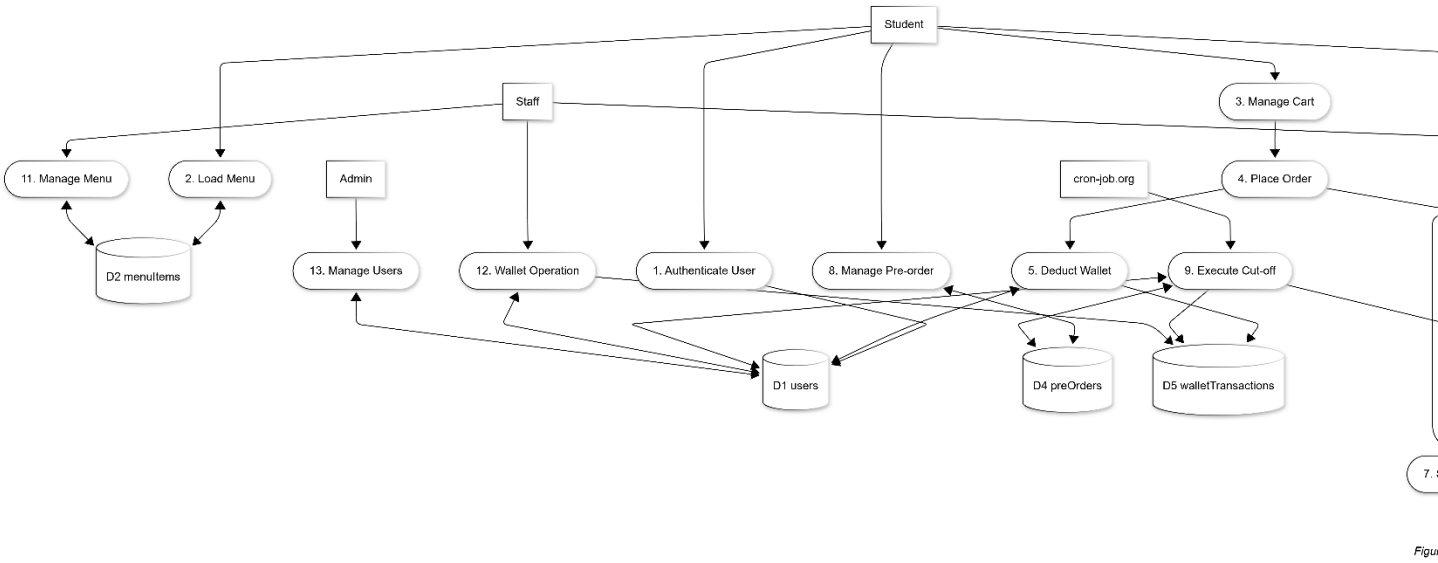


Figure 16 above shows the Level 1 DFD. The main processes are:

1. *Authenticate User* - Receives login credentials from Student/Staff, validates against Firebase Auth, returns session token.
2. *Load Menu* - Reads menuItems collection from Firestore; streams updates to student via snapshot listener.

3. *Manage Cart* - Receives add/remove/update actions from Student; maintains cart state in CartViewModel (device memory, no Firestore write until order placed).

4. *Place Order* - Receives confirmed cart and payment method from Student; writes order document to Firestore; if wallet payment, calls Deduct Wallet process atomically.

5. *Deduct Wallet* - Executes Firestore runTransaction() to deduct total from users.walletBalance and write a walletTransactions record simultaneously.

6. *Track Order* - Listens to orders collection for status changes; calculates estimated wait from count of Pending/Preparing orders ahead; streams updates to Student.

7. *Send Notification* - After the staff Android app writes status = Ready to Firestore, it sends an authenticated HTTP POST to the Supabase notify-order-ready Edge Function (fail-silent). The edge function reads the student's fcmToken from the users collection and dispatches an FCM push notification. This replaces the Firestore-trigger Cloud Function approach, which required the Firebase Blaze plan.

8. *Manage Pre-order* - Receives pre-order details from Student; writes preOrders document with status=scheduled; reads upcoming pre-orders list back to Student.

9. *Execute Cut-off* - Supabase Edge Function (process-cutoff) called by cron-job.org HTTP POST at 10:00 AM EAT (lunch) or 5:00 PM EAT (dinner): queries preOrders (mealSlot, status=scheduled, scheduledDate=today); for each match, calls Deduct Wallet via Firestore transaction; on insufficient funds, marks cancelled and sends FCM notification via Firebase Admin SDK (with preferRest: true - gRPC not supported in Deno runtime).

10. *Staff: Update Order Status* - Receives status change from Staff Android app; writes to orders collection; then app calls Send Notification (process 7).

11. *Staff: Manage Menu* - Receives availability toggle or CRUD action from Staff (Android) or Admin (web panel); updates menuItems collection; propagates to all student Menu screens via snapshot listener.

12. *Staff: Wallet Operation* - Receives student UID, amount, and operation type (credit or debit) from Staff; executes Firestore runTransaction() to update users.walletBalance; writes walletTransactions record including staffId for audit trail.

13. *Admin: Manage Users* - Admin creates staff account via web panel (Next.js API route → Firebase Admin SDK): creates Firebase Auth user, writes users document with role=staff and firstLogin=true. Admin views user directory from Firestore users collection.

iii. Level 2: Detailed DFD

The Level 2 DFD expands the two most complex processes - Place Order and Execute Cut-off - into their internal sub-steps.

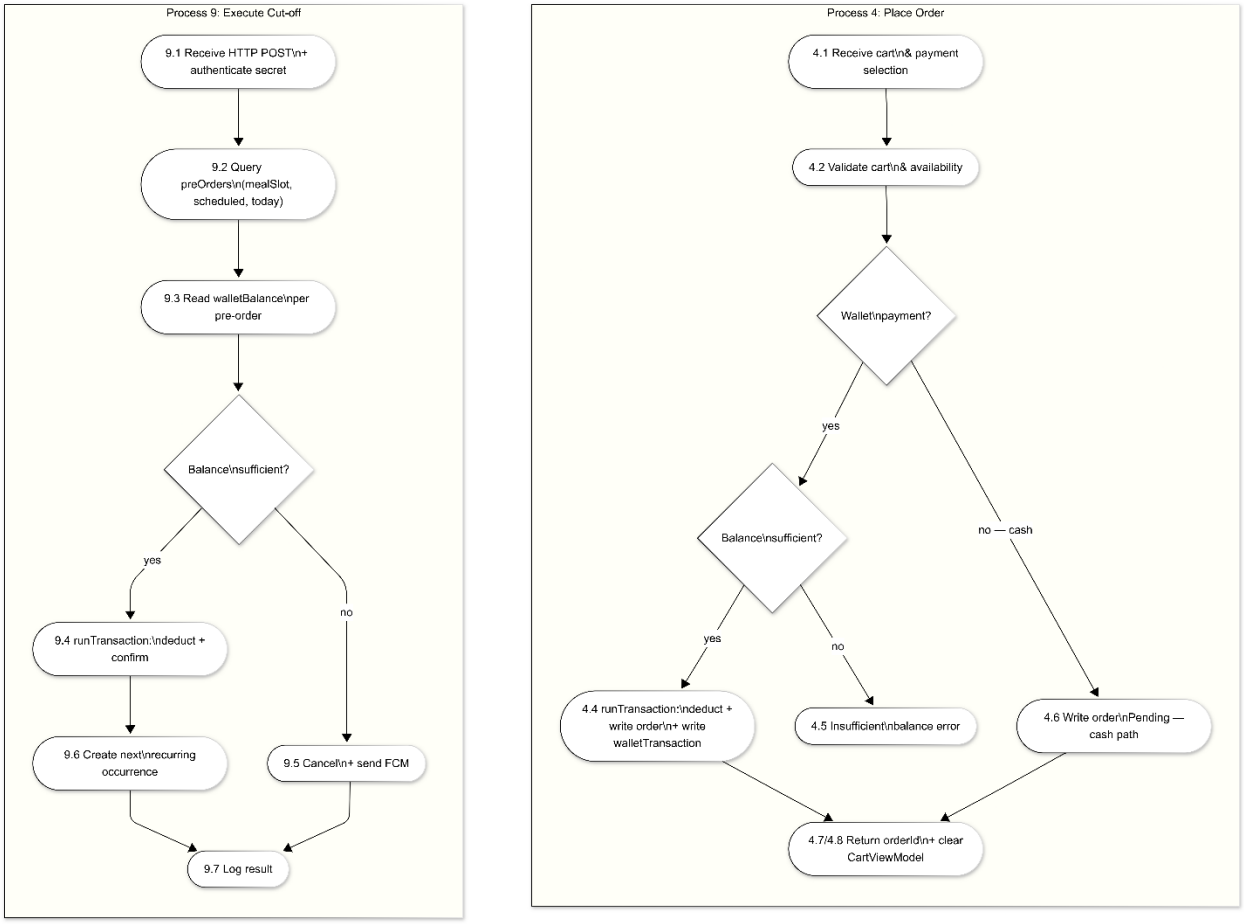


Figure 17: Level 2 DFD

Figure 17 above shows the Level 2 DFD. For *Place Order*: (1) Receive cart and payment selection from CartFragment; (2) Validate cart is non-empty and all items still available; (3) If payment = wallet, check walletBalance >= total; (4) If sufficient, call runTransaction() - deduct balance, write walletTransactions record, write order document with status=Pending; (5) If insufficient, return error to CartFragment without writing order; (6) If payment = cash, write order document directly with status=Pending; (7) Return order ID and confirmation to CartFragment; (8) Clear CartViewModel.

For *Execute Pre-order Cut-off*: (1) cron-job.org HTTP POST fires at the scheduled time (10:00 AM or 5:00 PM EAT) with mealSlot in the request body; (2) Supabase process-cutoff Edge Function authenticates the request using FUNCTIONS_SECRET header; (3) Firebase Admin SDK (REST mode - gRPC unsupported in Deno) queries preOrders where mealSlot=target, status=scheduled, scheduledDate=today; (4) For each pre-order: read users.walletBalance inside a Firestore transaction; (5) If balance >= total: runTransaction() - deduct balance from walletBalance, write walletTransactions document (with staffId omitted for system transactions), set preOrder.status=confirmed, send FCM confirmation via users.fcmToken; (6) If balance < total: set preOrder.status=cancelled, send FCM insufficient-funds notification via users.fcmToken; (7) If pre-order is recurring: create next occurrence document in preOrders for the next matching weekday; (8) Log execution result.

This Level 2 breakdown reveals the exact data stores touched in each critical operation and confirms that wallet atomicity is enforced at every deduction point through Firestore transactions.

5.1.5 Flowchart

A flowchart is a visual representation that outlines the step-by-step flow of processes within a system. It uses standardised symbols - ovals for start/end, rectangles for processes, diamonds for decisions, and arrows for flow direction. The USIU Cafeteria Ordering System has two distinct operational flows that are each represented as a separate flowchart: the regular order flow and the pre-order cut-off flow.

Flowchart A: Regular Order Flow

Figure 18a: Regular Order Flow

Figure 18a shows Flowchart A. The process begins when the Student launches the app and logs in. The system verifies credentials via Firebase Auth. If login fails, the user is prompted to retry. On success, the student is directed to the Menu screen where menu items are loaded from Firestore. The student browses the menu, selects items, and adds them to the cart. At checkout, the student

selects payment method - Wallet or Cash. If Wallet is selected, the system checks whether walletBalance >= order total. If insufficient, an error is shown and the student remains on the Cart screen. If sufficient (or Cash is selected), the order is placed - a Firestore transaction deducts the wallet balance (if applicable) and writes the order document with status=Pending. The student is navigated to the Orders screen where the live status tracker shows Pending → Preparing → Ready in real time. When the staff marks the order Ready, an FCM notification is sent to the student's device. The student collects the order at the counter.

Flowchart B: Pre-order Cut-off Flow

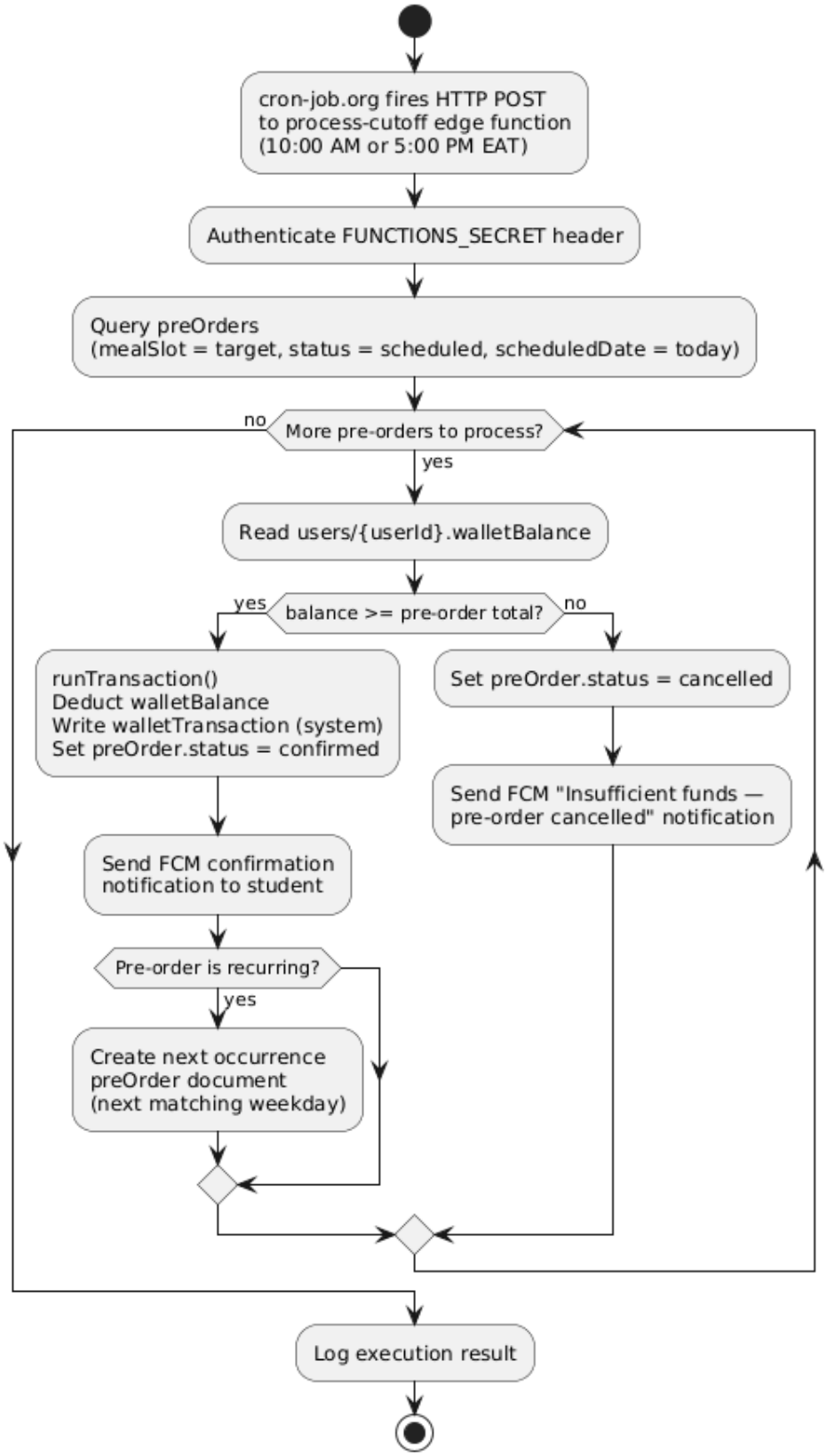


Figure 18b: Pre-order Cut-off Flow

Figure 18b shows Flowchart B. The process begins when the Firebase Cloud Function cron fires at the scheduled cut-off time (10:00 AM for lunch, 5:00 PM for dinner). The function queries all pre-orders with matching mealType, status=scheduled, and today's pickup date. For each pre-order found: the system reads the student's current walletBalance. If balance >= pre-order total, the system executes a Firestore transaction - deducts the balance, writes a walletTransactions record, and updates the pre-order status to confirmed. The cafeteria prepares the meal for the scheduled pickup time. If balance < pre-order total, the pre-order status is updated to cancelled and an FCM insufficient-funds notification is sent to the student's device. The student receives the notification and can top up their wallet and reschedule if they choose. The function logs its execution result and terminates until the next scheduled trigger.

5.1.6 Activity Diagram

An activity diagram is a type of behavioural diagram used in system modelling to represent the flow of activities and actions within a process. It focuses on how different tasks are carried out in sequence, the decisions made along the way, and how various activities connect to achieve a specific goal. For the USIU Cafeteria Ordering System, the activity diagram visualises the complete workflow across both student and staff roles, from login through to order collection, and separately for the pre-order scheduling lifecycle.

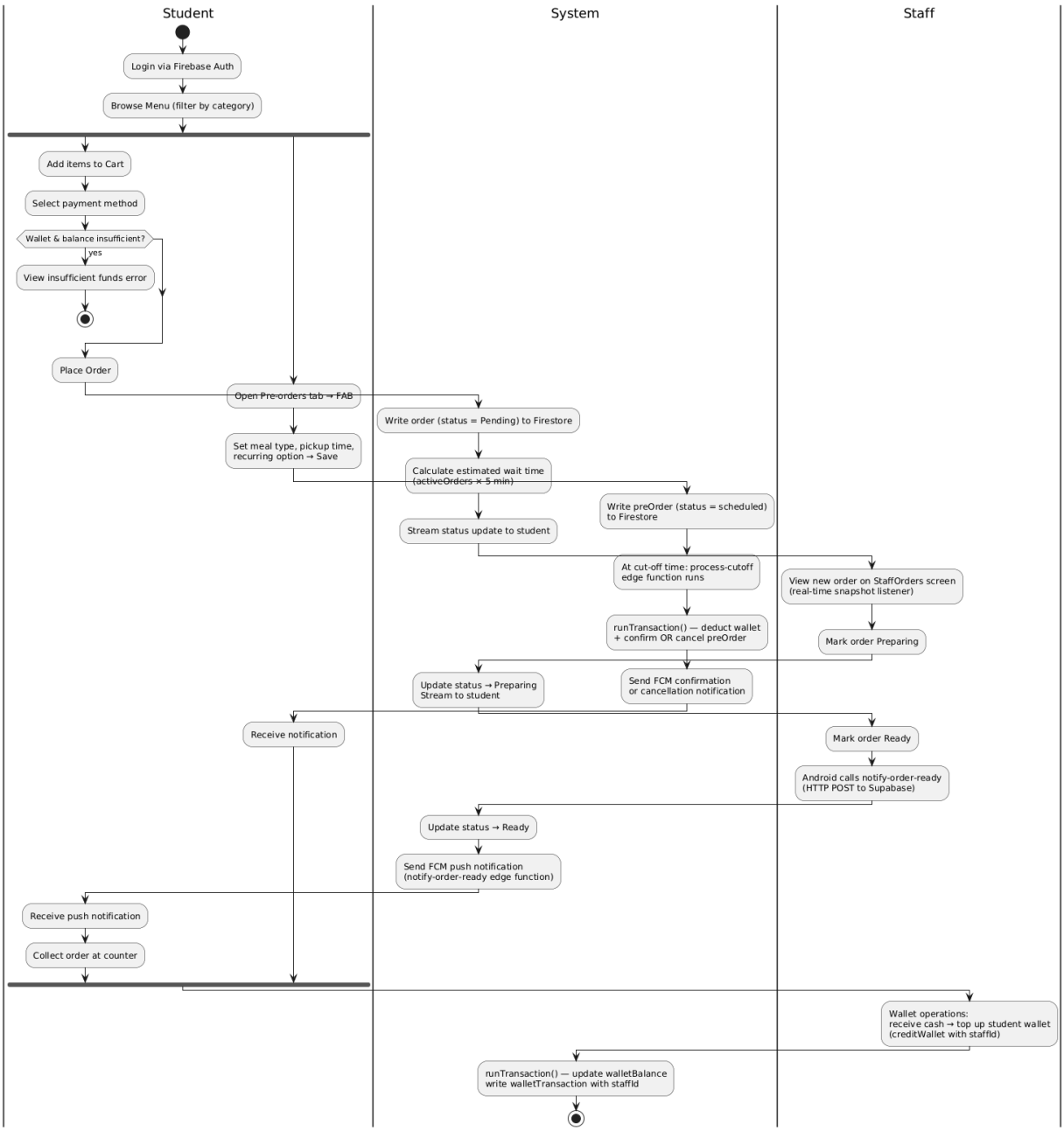


Figure 19: Activity Diagram — Order & Pre-order Lifecycle

Figure 19 above shows the activity diagram for the USIU Cafeteria Ordering System using swimlanes to separate the responsibilities of the Student, Staff, and System (Cloud Functions).

In the *Student swimlane*, activities begin with Login. On successful authentication, the student browses the menu, filtering by category if desired. The student selects items and adds them to the cart, adjusts quantities, and selects a payment method. If the chosen method is Wallet, a balance

check occurs - if funds are insufficient, the student is returned to the cart. Otherwise, the order is placed and the student waits on the Orders screen, monitoring the live status tracker and the estimated wait time indicator. When the Ready notification arrives, the student collects their meal. For the pre-order path, the student instead navigates to the Pre-orders screen, enters meal details, selects a pickup time, optionally sets a recurring day, and saves the pre-order.

In the *Staff swimlane*, activities begin with Login (staff role). The staff views the incoming orders list in real time. On receiving a new order, the staff marks it as Preparing and begins preparation. When the meal is ready, the staff marks it as Ready. For menu management, staff can toggle item availability at any point. For wallet management, staff receive cash from a student, input the amount and student ID, and submit the top-up.

In the *System swimlane*, activities include: streaming menu updates to all connected student devices when availability changes; updating the order status display on the student's Orders screen when staff writes a status change; calculating estimated wait time on each incoming order update; and executing the cut-off cron job - scanning scheduled pre-orders, deducting wallets, and sending notifications - at the designated times.

The activity diagram demonstrates that the system operates across three concurrent actors, with the System swimlane bridging the Student and Staff swimlanes through real-time data propagation.

5.1.7 Sequence Diagram

A sequence diagram is a type of interaction diagram used in system design to show how objects and components interact with each other over time. It focuses on the order of messages exchanged between different entities to accomplish a specific function. For the USIU Cafeteria Ordering System, two sequence diagrams are presented: one for the regular order flow and one for the pre-order cut-off execution.

Sequence Diagram A: Regular Order Placement

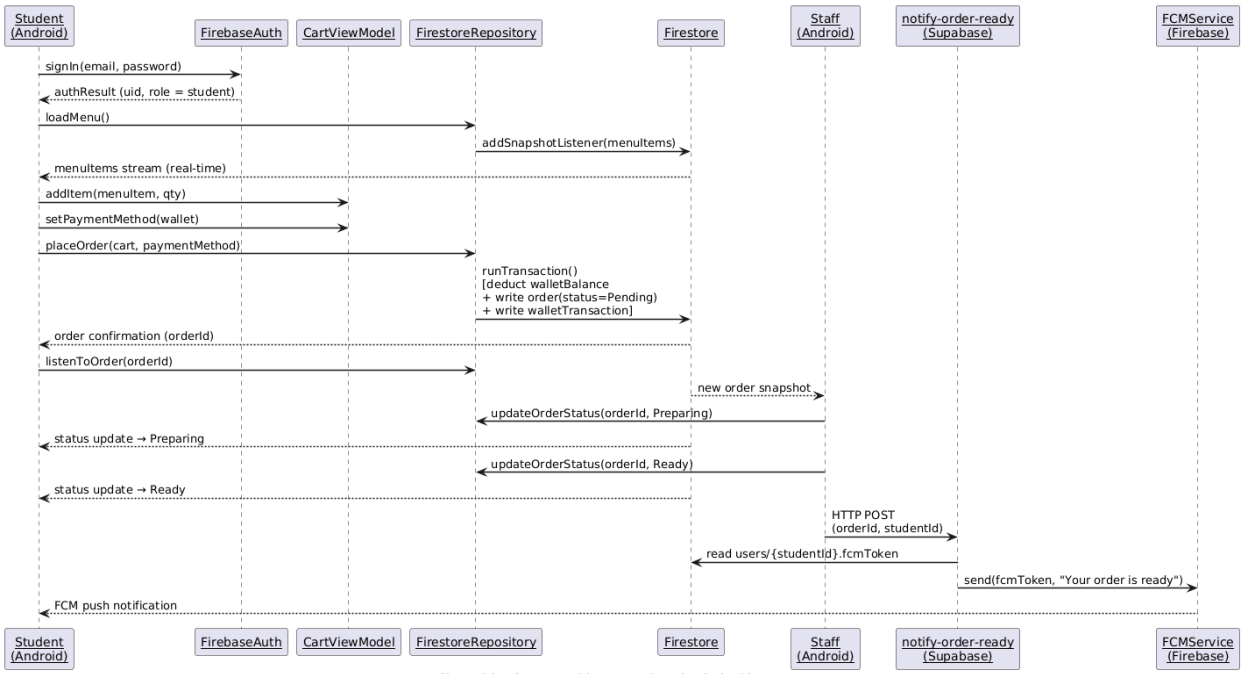


Figure 20a: Sequence Diagram — Regular Order Placement

Figure 20a shows the sequence of interactions for a regular order. The participants are: Student (Android client), FirebaseAuth, CartViewModel, FirestoreRepository, Firestore, StaffOrdersFragment, and FCMService.

1. Student → FirebaseAuth: signIn(email, password)
2. FirebaseAuth → Student: authResult (uid, role)
3. Student → FirestoreRepository: loadMenu()
4. FirestoreRepository → Firestore: addSnapshotListener(menuItems)
5. Firestore → Student: menuItems stream
6. Student → CartViewModel: addItem(menuItem, quantity)
7. Student → CartViewModel: setPaymentMethod(wallet)
8. Student → FirestoreRepository: placeOrder(cart, paymentMethod)
9. FirestoreRepository → Firestore: runTransaction() [deduct walletBalance + write order(status=Pending)]
10. Firestore → Student: order confirmation (orderId)
11. Student → FirestoreRepository: listenToOrder(orderId)

12. Firestore → StaffOrdersFragment: new order notification (snapshot)
13. Staff → FirestoreRepository: updateOrderStatus(orderId, Preparing)
14. Firestore → Student: status update (Preparing)
15. Staff → FirestoreRepository: updateOrderStatus(orderId, Ready)
16. Firestore → FCMService: onOrderStatusChanged trigger
17. FCMService → Student: FCM push notification ("Your order is ready")
18. Firestore → Student: status update (Ready)

Sequence Diagram B: Pre-order Cut-off

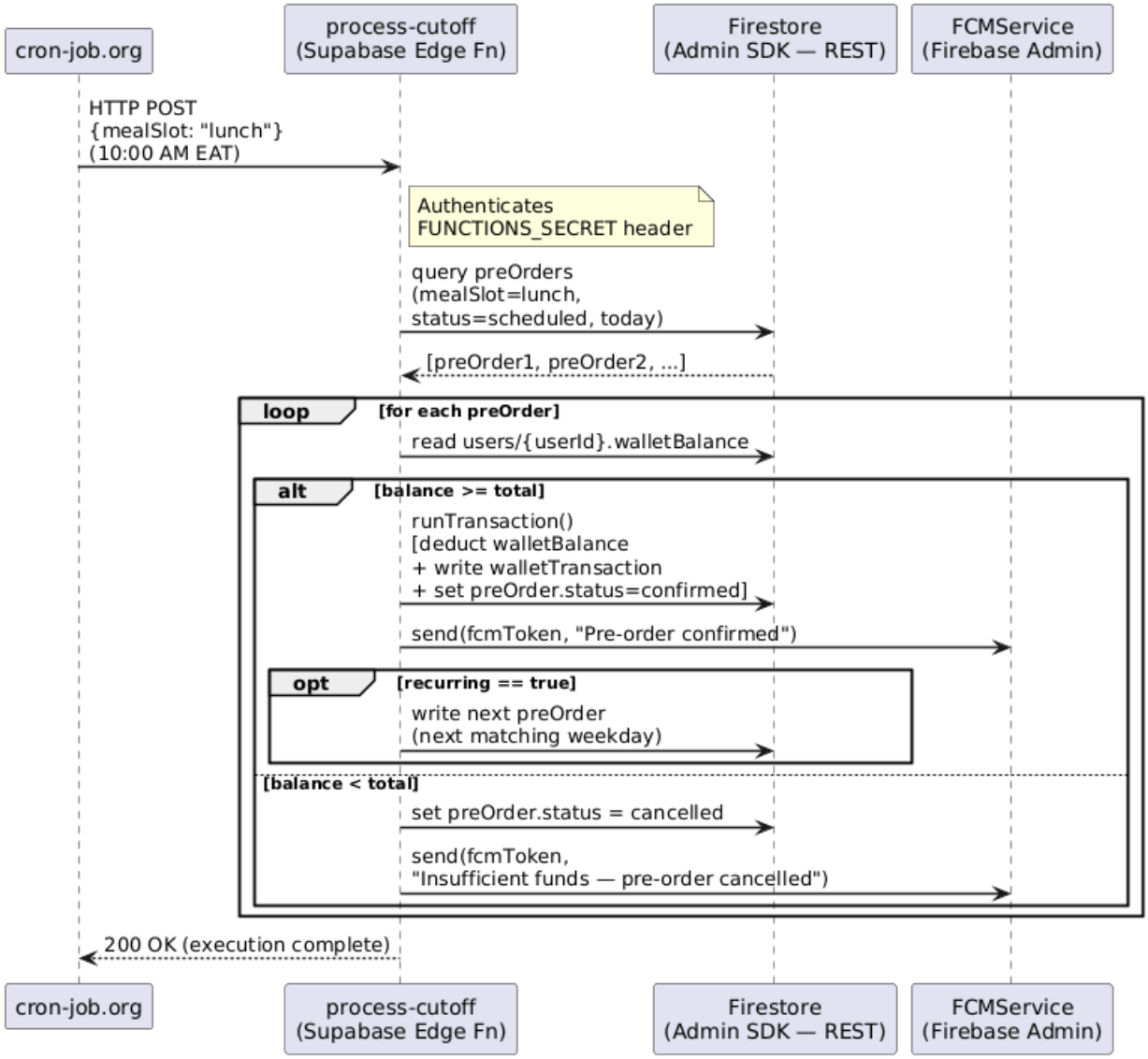


Figure 20b: Sequence Diagram — Pre-order Cut-off Execution

Figure 20b shows the sequence for the pre-order cut-off. The participants are: CloudScheduler, lunchCutoffFunction, Firestore, and FCMService.

1. CloudScheduler → lunchCutoffFunction: trigger (10:00 AM)
2. lunchCutoffFunction → Firestore: query preOrders (mealType=lunch, status=scheduled, today)
3. Firestore → lunchCutoffFunction: [preOrder1, preOrder2, ...]

4. For each preOrder: a. lunchCutoffFunction → Firestore: read users/{userId}.walletBalance b. If balance >= total:

   - lunchCutoffFunction → Firestore: runTransaction() [deduct walletBalance + write walletTransaction + update preOrder.status=confirmed] c. If balance < total:
   - lunchCutoffFunction → Firestore: update preOrder.status=cancelled
   - lunchCutoffFunction → FCMService: sendNotification(deviceToken, "Insufficient funds - pre-order cancelled")

5. lunchCutoffFunction → CloudScheduler: execution complete

5.1.8 Class Diagram

A class diagram is a type of structural diagram in system design that illustrates the blueprint of an object-oriented system by showing the system's classes, their attributes, methods, and the relationships among them. It provides a static view of the system, helping developers understand how components interact and depend on one another. For the USIU Cafeteria Ordering System, the class diagram represents the Android application's Java class structure. The system also includes a separate web admin panel (Next.js, TypeScript) whose components are not represented here as they are not Java classes; a note on this is included at the end of this section.

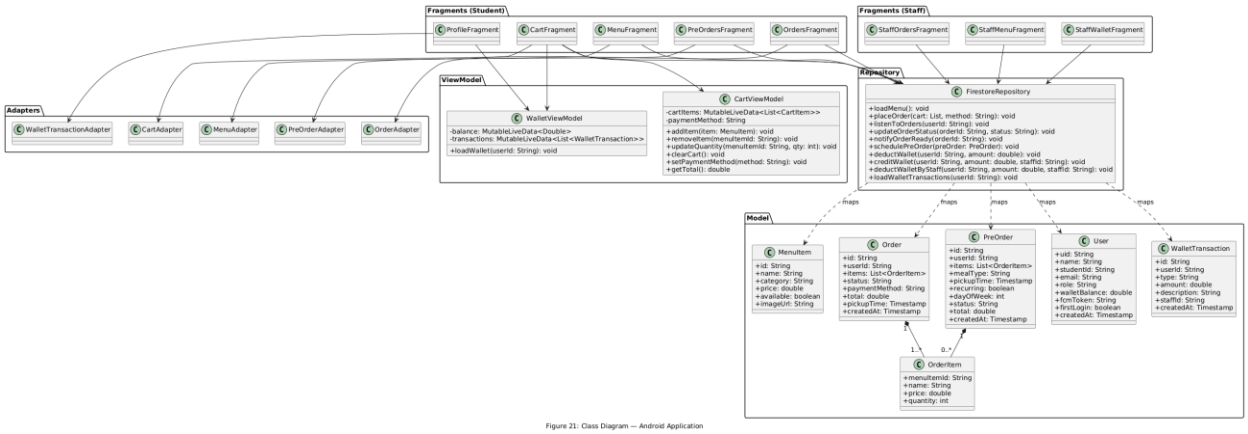


Figure 21: Class Diagram — Android Application

Figure 21 above shows the class diagram for the USIU Cafeteria Ordering System. The key classes are grouped by responsibility:

Model Classes (Data)

- *MenuItem:* id, name, category, price, available, imageUrl
- *Order:* id, userId, items (List<OrderItem>), status, paymentMethod, total, pickupTime, createdAt
- *OrderItem:* menuItemId, name, price, quantity
- *PreOrder:* id, userId, items (List<OrderItem>), mealType, pickupTime, recurring, dayOfWeek, status, total, createdAt
- *User:* uid, name, studentId, email, role, walletBalance, fcmToken, firstLogin, createdAt
- *WalletTransaction:* id, userId, type, amount, description, staffId, createdAt

Fragment Classes (UI)

- *MenuFragment:* Observes menuItems via FirestoreRepository; uses MenuAdapter and CategoryAdapter; navigates to CartFragment
- *CartFragment:* Observes CartViewModel; displays wallet balance; calls FirestoreRepository.placeOrder()
- *OrdersFragment:* Observes active orders via FirestoreRepository; calculates estimatedWait(); uses OrderAdapter
- *PreOrdersFragment:* Reads/writes preOrders via FirestoreRepository; uses PreOrderAdapter
- *ProfileFragment:* Observes WalletViewModel; displays balance and transaction history
- *StaffOrdersFragment:* Observes all orders via FirestoreRepository; calls updateOrderStatus()
- *StaffMenuFragment:* Calls FirestoreRepository.updateItemAvailability()
- *StaffWalletFragment:* Calls FirestoreRepository.creditWallet() or FirestoreRepository.deductWallet(); records staffId on each transaction

ViewModel Classes (State)

- *CartViewModel:* MutableLiveData<List<CartItem>>; addItem(), removeItem(), updateQuantity(), clearCart(), setPaymentMethod()
- *WalletViewModel:* MutableLiveData<Double> balance; MutableLiveData<List<WalletTransaction>>; loads from FirestoreRepository

Adapter Classes (RecyclerView)

- *MenuAdapter, CategoryAdapter, CartAdapter, OrderAdapter, PreOrderAdapter, WalletTransactionAdapter* Each extends RecyclerView.Adapter; binds the relevant model class to a ViewHolder layout

Repository Class (Data Access)

- *FirestoreRepository:* Central data access object; all Firestore reads and writes pass through this class. Key methods: loadMenu(), placeOrder(), listenToOrders(), updateOrderStatus(), notifyOrderReady() (HTTP POST to Supabase notify-order-ready edge function), schedulePreOrder(), deductWallet() (runTransaction), creditWallet() (runTransaction), deductWalletByStaff() (runTransaction, writes staffId), loadWalletTransactions()

Relationships: Fragments depend on FirestoreRepository and ViewModel classes. ViewModels hold LiveData observed by Fragments. Adapters are owned by Fragments. FirestoreRepository maps Firestore documents to model class instances using Firestore's toObject() method.

Note - Admin Web Panel (out of scope for this Java class diagram): The system includes a separate web-based admin panel built with Next.js and TypeScript. Its components - login page, dashboard pages for menu management and staff account creation, Next.js API routes using the Firebase Admin SDK, and Tailwind CSS UI - are TypeScript/React components and are not represented in this Java class diagram. The admin panel interacts with the same Firestore database and Firebase Authentication instance as the Android app, governed by the same Firestore security rules.

5.1.9 Wireframes

The wireframes for the USIU Cafeteria Ordering System serve as the visual blueprint for all application interfaces. They outline how each screen will be structured, what components will appear, and how users will navigate between functions. At this stage the focus is on layout and interaction flow rather than final colours or visual polish. The wireframes cover two applications: the Android mobile app (five student screens and three staff screens) and the web-based admin panel (four web pages). Together they ensure that every feature - menu browsing, cart checkout, order tracking, pre-order scheduling, wallet management, staff order processing, and admin account and menu control - is logically organised before implementation begins.

[Figure 22: Wireframes - Android app (Menu, Cart, Orders, Pre-orders, Profile/Wallet) and web admin panel (Login, Menu Management, Staff Creation, Users Directory)]
Figure 22 above represents the wireframes for the USIU Cafeteria Ordering System.

Screen 1: Menu

The Menu screen is the home screen for student users. A horizontal tab bar at the top allows filtering by category: All, Mains, Snacks, Drinks, and Specials. Below the tabs, a vertical scrollable list of item cards fills the main content area. Each card displays the item image, name, price, and an availability chip (green "Available" or red "Sold Out"). A floating action button (FAB) in the bottom-right corner shows the current cart item count and navigates to the Cart screen. A search icon in the top app bar expands an inline search field for text-based item lookup.

Screen 2: Cart

The Cart screen displays all items currently in the cart in a scrollable list. Each row shows the item name, unit price, and +/– quantity controls, with a remove button. A subtotal updates dynamically below the list. A payment method toggle allows the student to switch between Wallet (showing current balance) and Cash. If Wallet is selected and the balance is insufficient, a warning indicator appears below the toggle. A prominent "Place Order" button at the bottom is disabled if the cart is empty or if Wallet is selected with insufficient funds.

Screen 3: Orders

The Orders screen has two sections. The top section shows the active order (if one exists) with a three-step horizontal status bar: Pending → Preparing → Ready. The current step is highlighted. An estimated wait time chip (e.g, "~10 min") appears below the status bar, updating in real time as the queue changes. The bottom section is a scrollable history list of past completed orders, each showing date, items summary, total, and payment method. Tapping a past order expands it to show full details and a "Reorder" button.

Screen 4: Pre-orders

The Pre-orders screen shows a list of upcoming scheduled pre-orders, each displayed as a card with meal type, pickup time, total, recurring badge (if set), and a cancel button. A FAB opens the Schedule Pre-order dialog. The dialog contains: a meal type selector (Breakfast / Lunch / Dinner), a date and time picker for the pickup slot, an item selection interface (same as the menu), a recurring toggle with a day-of-week selector, and a wallet balance preview showing post-deduction balance at cut-off.

Screen 5: Profile / Wallet

The Profile/Wallet screen opens with a prominent wallet balance card at the top showing the current balance and a note indicating that top-ups are done at the cafeteria counter. Below the balance card is a scrollable list of recent wallet transactions (credits and debits) with date, description, and amount. Below transactions is the student's account details section (name, student ID, email) and a logout button.

Web Admin Panel (Sixth Interface)

The admin panel is a browser-based application accessible only to users with the admin role. It consists of four pages:

Admin Login Page: A centred login form with email and password fields and a "Sign In" button. On successful login, the system checks the role field - if not admin, the user is redirected. If firstLogin is true, the user is redirected to the change password page before accessing the dashboard.

Dashboard - Menu Management: A full-width data table listing all menu items with columns for name, category, price, availability toggle (switch), and image URL. An "Add Item" button above the table opens an inline form or modal with fields for name, description, category (dropdown), price, imageUrl, and availability. Each row has Edit and Delete action buttons. Changes write to the Firestore menuItems collection via the Firebase Client SDK.

Dashboard - Staff Account Creation: A simple form with fields for full name, email, and temporary password, and a "Create Staff Account" button. On submission, the Next.js API route calls the Firebase Admin SDK to create a Firebase Auth user and write a Firestore users document with role=staff and firstLogin=true. A success message shows the new account's email.

Dashboard - Users Directory: Two separate tables on one page. The first table lists all students (name, student ID, email, wallet balance, joined date). The second table lists all staff and admin accounts (name, email, role, firstLogin status). Both tables are read from the Firestore users collection, filtered by role, via the Firebase Client SDK.

5.2 Functional Requirements

Functional requirements define the specific actions and features that the Cafeteria Ordering System must provide to meet the needs of students, staff, and administrators.

Student Functional Requirements

- *FR1:* The system shall allow students to browse the cafeteria menu, filtered by category (breakfast, lunch, dinner) and item availability.
- *FR2:* The system shall allow students to add items to a cart, adjust quantities, and review the order before confirming.
- *FR3:* The system shall allow students to place a regular order and choose to pay via the in-app wallet or declare cash payment at the counter.
- *FR4:* The system shall display a real-time estimated wait time for each order, calculated from the number of pending and preparing orders ahead in the queue and the average preparation time per item.

- *FR5:* The system shall allow students to place a smart pre-order for a future meal slot (lunch or dinner), subject to cut-off times (10:00 AM for lunch, 5:00 PM for dinner).
- *FR6:* The system shall allow students to set a pre-order as recurring on selected days of the week (e.g, every Thursday at 12:30).
- *FR7:* The system shall allow students to view the status of their active and past orders in real time.
- *FR8:* The system shall allow students to view their wallet balance and a transaction history of top-ups and deductions.

Staff Functional Requirements

- *FR9:* The system shall allow staff to view all incoming orders and update their status (Pending → Preparing → Ready → Collected).
- *FR10:* The system shall allow staff to mark a cash order as paid when the student collects and pays at the counter.
- *FR11:* The system shall allow staff to manage the menu - add, edit, enable, or disable menu items and update daily availability.
- *FR12:* The system shall allow staff to credit (top up) or debit (deduct) a student's wallet balance by entering the student's ID and the amount; each transaction shall record the staff member's ID (staffId) for accountability.

Admin Functional Requirements

- *FR13:* The system shall allow administrators to create staff accounts via the web admin panel; each new staff account shall be assigned a temporary password, and the staff member shall be required to change it on first login before accessing any operational screens.
- *FR14:* The system shall allow administrators to add, edit, delete, and toggle the availability of menu items via the web admin panel, with changes reflected immediately on all student Menu screens through Firestore snapshot listeners.

- *FR15:* The system shall allow administrators to view a directory of all users, with students and staff/admin accounts displayed in separate tables.

System Functional Requirements

- *FR16:* The system shall automatically process all confirmed pre-orders at the respective cut-off time via the Supabase process-cutoff Edge Function (called by cron-job.org at 10:00 AM EAT for lunch and 5:00 PM EAT for dinner): deduct the order total from the student's wallet if funds are sufficient, or cancel the pre-order and send an FCM push notification if the wallet balance is insufficient.
- *FR17:* The system shall send FCM push notifications to students when their order status changes to Ready (via the Supabase notify-order-ready Edge Function, called by the Android app after the staff status update) and when a pre-order is confirmed or cancelled due to insufficient funds (via the process-cutoff Edge Function).
- *FR18:* The system shall enforce secure authentication via Firebase Authentication, ensuring that students (Android app), staff (Android app), and administrators (web panel) access only the data and screens appropriate to their role, as enforced by Firestore security rules using isStudent(), isStaff(), isAdmin(), and isPrivileged() helper functions.
- *FR19:* The system shall perform all wallet deductions atomically using Firestore runTransaction() to prevent race conditions or double-spending.
- *FR20:* The system shall maintain a complete, timestamped audit trail of all wallet transactions (top-ups and deductions) in the walletTransactions Firestore collection, including the staffId of the staff member who performed each transaction.

5.3 Non-Functional Requirements

Non-functional requirements describe the quality attributes and constraints that the Cafeteria Ordering System must satisfy beyond its core features.

- *NFR1: Performance Requirements* The system shall display the current menu and cart within 2 seconds on a stable mobile data or Wi-Fi connection. Real-time order status

updates shall be reflected in the app within 3 seconds of a staff status change. The estimated wait time calculation shall execute entirely on the client using a cached Firestore query result and shall complete within 1 second.

- *NFR2: Security Requirements* All users must authenticate via Firebase Authentication before accessing any system function. Firestore security rules shall enforce role-based access using three helper functions - isStudent(), isStaff(), and isAdmin() - with an isPrivileged() helper for shared staff/admin operations: students may only read and write their own orders, pre-orders, and wallet records; staff may read and update all orders and perform wallet top-ups and deductions; admin may read and write all collections. No client may directly modify another user's wallet balance without a server-side Firestore transaction. Supabase Edge Functions authenticate via a shared FUNCTIONS_SECRET header; the admin panel's privileged operations (staff account creation) execute via Next.js server-side API routes using the Firebase Admin SDK, never exposing admin credentials to the browser. Wallet deductions use Firestore runTransaction() to guarantee atomicity and prevent double-spending.

- *NFR3: Usability Requirements* The application shall follow Material Design 3 guidelines with USIU branding (navy #002147, gold #CFB991) to ensure a consistent and familiar experience. Navigation between the five main screens (Menu, Cart, Orders, Pre-orders, Profile/Wallet) shall require no more than one tap from any screen. The pre-order form shall clearly display the applicable cut-off time and wallet-only payment constraint so that students are not surprised by automatic deductions.

- *NFR4: Reliability Requirements* The system shall rely on Firebase's managed infrastructure, which provides 99.95% uptime SLA, to store all order and wallet data. Pre-order cut-off processing shall be performed by the Supabase process-cutoff Edge Function, triggered by cron-job.org at the scheduled times, rather than client devices - ensuring that deductions occur even when no student has the app open. All Firestore writes shall use offline persistence so that students can browse the menu and build a cart without an active connection, with synchronisation occurring when connectivity is restored.

- *NFR5: Scalability* The system shall use Firebase Firestore, a horizontally scalable NoSQL database, allowing the number of concurrent users, menu items, and orders to grow without schema migrations or infrastructure changes. Adding new meal slots, menu categories, or staff accounts shall require only Firestore document inserts or Firebase Authentication role assignments, with no changes to application code.

- *NFR6: Maintainability* All Firestore interactions in the Android app shall be centralised in a FirestoreRepository class, so that query logic can be updated in one place without modifying Activity or Fragment code. Business logic for pre-order cut-off shall reside exclusively in the Supabase process-cutoff Edge Function (TypeScript), making it independently deployable, testable via HTTP POST, and updatable without rebuilding the Android APK. The admin panel's privileged Firebase operations shall reside in Next.js API routes, keeping admin SDK credentials off the client.

- *NFR7: Compliance* The system shall not store any payment card or M-Pesa credentials. Wallet top-ups are performed in person (cash at the counter), and all financial records are maintained within the university's own Firestore project, reducing data-sovereignty and regulatory risk.

5.4 Conclusion

The Cafeteria Ordering System is a project designed to address the real challenges of food ordering and queue management at the United States International University Africa (USIU-Africa) cafeteria. This chapter has presented the complete system design, covering the system models, functional requirements, and non-functional requirements that together define both what the system must do and the quality standards it must meet.

The system models section documented the architecture of a three-tier Android and Firebase solution and illustrated its behaviour through a comprehensive set of diagrams: use case, entity-relationship, data flow, flowchart, activity, sequence, class, and wireframe diagrams. These diagrams confirm that the system is well-structured, with clear separation between the student-facing mobile application, the Firebase backend services, and the staff management interface.

The functional requirements capture seventeen specific capabilities spanning student ordering (browse, cart, regular order, pre-order, recurring schedule), real-time estimated wait time, staff management (order processing, menu management, wallet top-up), and system-level automation (pre-order cut-off via Cloud Functions, FCM push notifications, atomic Firestore transactions, and a complete wallet audit trail). The non-functional requirements establish the performance, security, usability, reliability, scalability, maintainability, and compliance standards that the implementation must satisfy.

In conclusion, the analysis and design presented in this chapter provide a solid and feasible blueprint for the Cafeteria Ordering System. The combination of a modern Android front end, a scalable Firebase backend, and automated pre-order processing through Cloud Functions distinguishes this system from conventional point-of-sale or manual queuing approaches. With this design in place, the implementation phase, covered in the following chapter, can proceed with clarity and confidence.

6 CHAPTER 6: IMPLEMENTATION

The implementation phase of the USIU Cafeteria Ordering System represented the transformation of the design specifications established in Chapter 5 into two fully operational, production-ready deliverables: a native Android mobile application serving students and cafeteria staff, and a web-based admin panel for system administrators. This chapter documents the development process, the technical structures adopted, and the integration patterns that govern how all components communicate, from the user interface through to the cloud backend. The Android application was built in Java using XML-based layouts and Material Design 3 components, following an MVVM architecture with a centralised FirestoreRepository for all data access. The admin panel was built as a separate Next.js 14 application using TypeScript and Tailwind CSS, sharing the same Firebase project. The backend relies on Firebase Firestore, Firebase Authentication, and Firebase Cloud Messaging, supplemented by Supabase Edge Functions for scheduled pre-order processing and push notification dispatch. All infrastructure is hosted on free-tier plans, resulting in a production-ready system at zero operational cost.

6.1 Introduction

The implementation phase of the USIU Cafeteria Ordering System represented the crucial transformation of the theoretical design specifications and comprehensive requirements documentation into fully operational, production-ready software. This development stage encompassed the systematic construction of two distinct client applications - a native Android mobile application and a web-based admin panel - along with the configuration of a Firebase cloud backend and two Supabase Edge Functions that handle automated pre-order processing and push notification dispatch. The primary objective was to deliver a robust, intuitive, and feature-complete cafeteria ordering solution addressing the needs of three user roles: students who browse the menu, place orders, and manage pre-orders and wallet balances; cafeteria staff who process incoming orders and handle student wallet operations; and administrators who manage the menu catalogue and create staff accounts. Special emphasis was placed on ensuring that all payment operations - wallet deductions, top-ups, and pre-order charges at cut-off time -

execute atomically using Firestore transactions, preventing any inconsistency in wallet balances under concurrent access.

The implementation prioritised simplicity and reliability over complexity. The Android app follows a single-activity architecture with a BottomNavigationView and fragment-based navigation using hide/show transactions to preserve scroll state. All Firestore access is centralised in a single FirestoreRepository class, and UI state is managed through ViewModel and LiveData classes following the MVVM pattern. The admin panel uses Next.js 14 App Router with server-side Firebase Admin SDK calls to securely create staff accounts and manage Firestore data. Both applications share the same Firebase project and Firestore database, governed by a unified set of security rules that enforce role-based access at the document level.

6.2 Frontend Implementation

The frontend of the USIU Cafeteria Ordering System comprises two distinct client applications. The primary deliverable is a native Android application built in Java with XML layouts, serving both students and cafeteria staff through role-specific navigation. The secondary deliverable is a web-based admin panel built with Next.js 14 and TypeScript, accessible via a desktop browser for system administrators. Both applications adhere to Material Design 3 principles and USIU branding guidelines, incorporating the university's signature navy (#002147) and gold (#CFB991) colour palette throughout all interface elements.

i. Authentication and Role Routing

Authentication is handled centrally by Firebase Authentication using email and password credentials. The LoginActivity submits user credentials via Firebase Auth's signInWithEmailAndPassword method. On successful authentication, the app immediately reads the users/{uid} Firestore document to retrieve the role field. If the role is "student", the user is directed to MainActivity with the student bottom navigation configuration. If the role is "staff", the user is directed to StaffMainActivity with the staff navigation. This role check prevents students from accessing staff screens and vice versa without requiring server-side session management. Students self-register through the RegisterActivity, which creates a Firebase Authentication user and simultaneously writes a Firestore document to the users collection with role: "student" and walletBalance: 0.0. Staff accounts cannot be self-registered; they are created

exclusively by administrators via the web admin panel, which calls a Firebase Admin SDK API route to provision both the Auth user and the Firestore document. When a staff or admin user logs in for the first time, the firstLogin field on their Firestore document is true. The app detects this and redirects to ChangePasswordActivity before granting access to any operational screen, enforcing a mandatory password change on initial access.

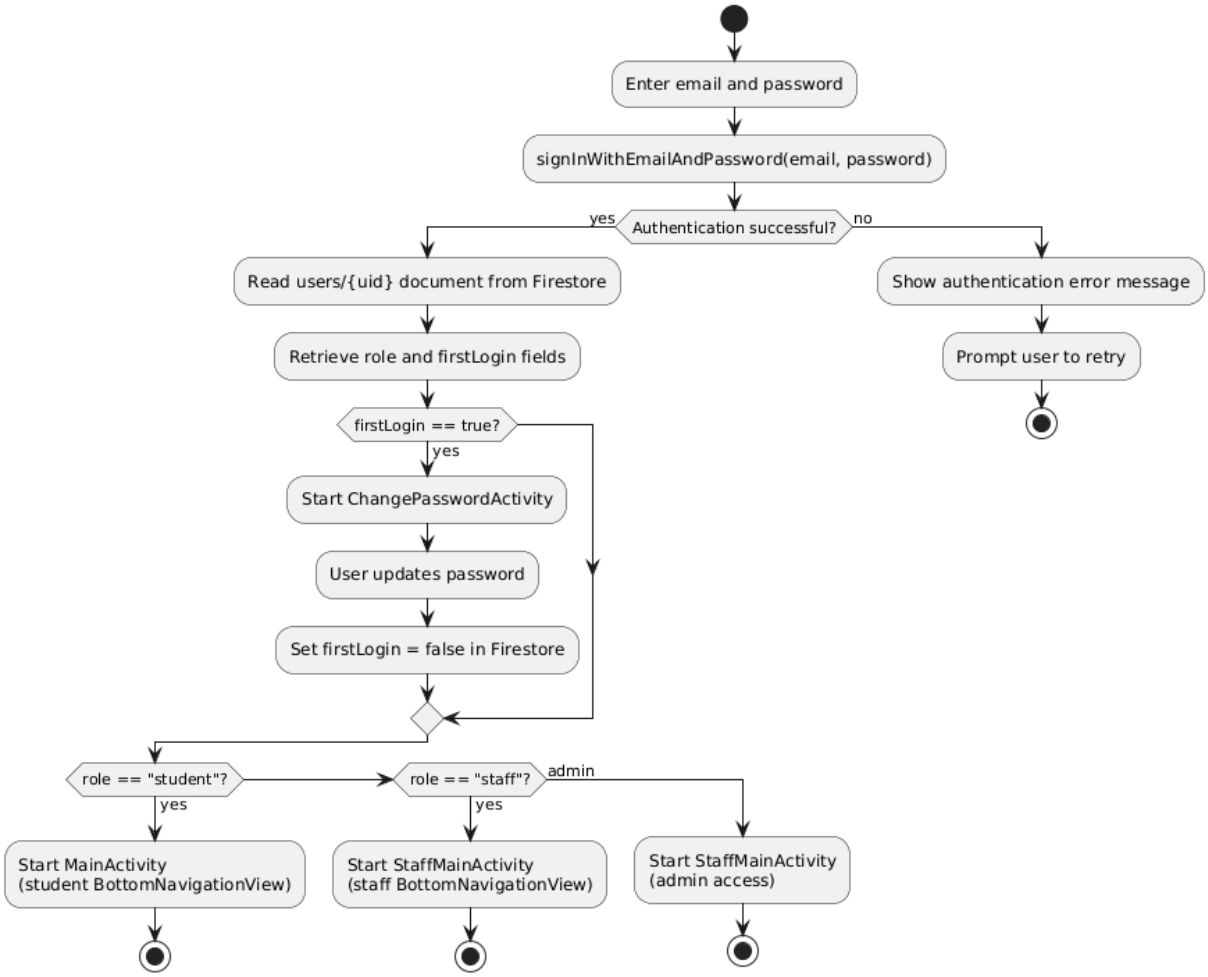


Figure 23: LoginActivity — Firebase Authentication and Role-based Routing

Figure 23 illustrates the authentication flow. On login, the signInWithEmailAndPassword callback reads the user's Firestore document to determine role and firstLogin status. If firstLogin is true, the activity starts ChangePasswordActivity and finishes itself, preventing back-navigation to the pre-change state. Once the password is updated, firstLogin is set to false in Firestore, and the user is routed to the appropriate main activity.

ii. Student Interface - Menu and Cart

The student interface is hosted within MainActivity, which manages five fragments through a BottomNavigationView. Navigation uses fragment hide/show transactions rather than replace transactions; this preserves the RecyclerView scroll position in the Menu and Orders tabs when the user switches between screens, eliminating the need to re-query Firestore on every tab switch. The MenuFragment serves as the home screen for student users. It presents a horizontal ChipGroup at the top of the screen allowing the user to filter items by category: All, Breakfast, Lunch, and Dinner. Below the chips, a RecyclerView backed by a MenuAdapter displays item cards. Each card shows the item image (loaded asynchronously using Glide with a placeholder drawable), the item name, price in KES, and a short description. Items with available == false render with a "Sold Out" chip and a disabled, greyed-out "Add to Cart" button, ensuring that unavailable items are visible but not orderable. Tapping the "Add to Cart" button dispatches the item to CartViewModel and shows a snackbar confirming the addition. Category filtering is performed in-memory on the already-loaded LiveData list, requiring no additional network call.

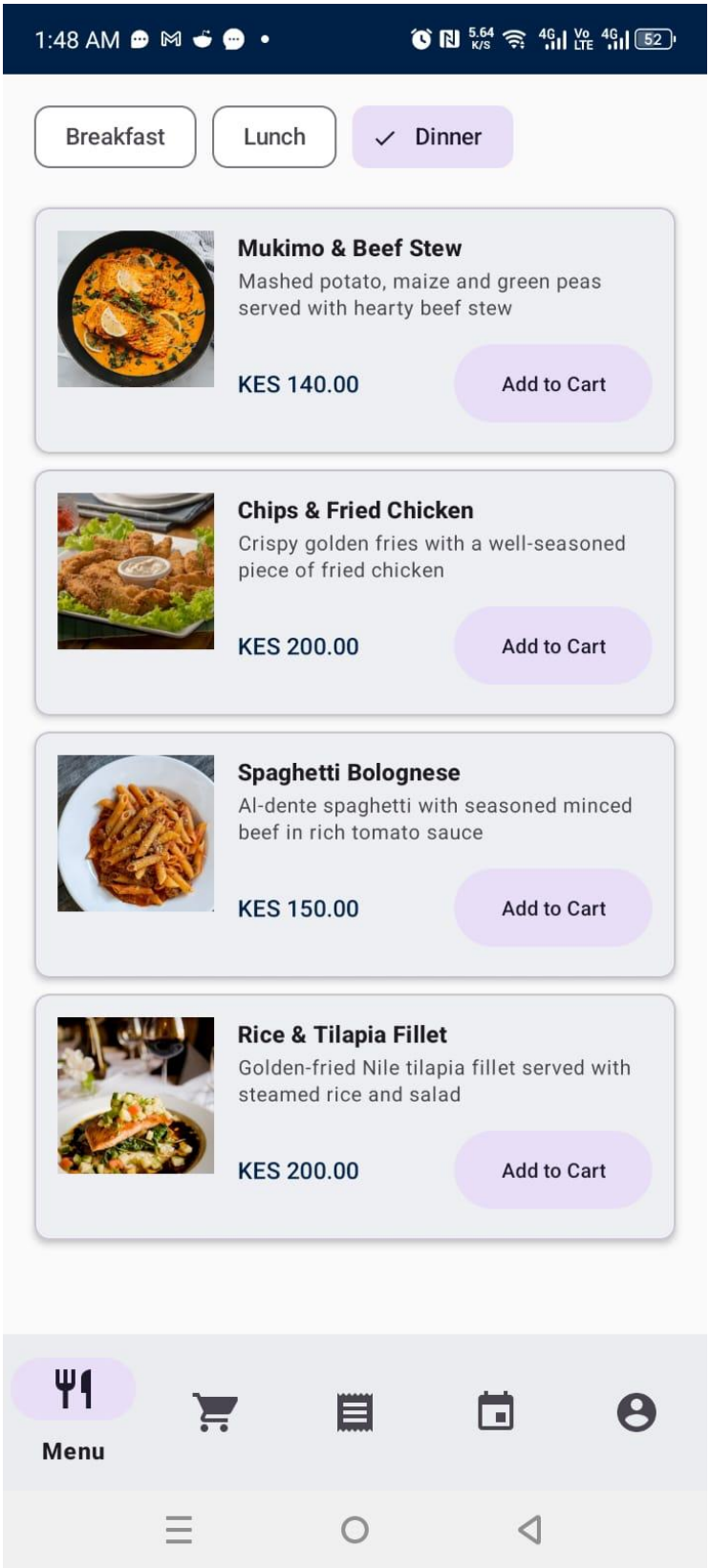

1:48 AM
Breakfast
Lunch
Dinner
Mukimo & Beef Stew
Mashed potato, maize and green peas served with hearty beef stew
KES 140.00
Add to Cart
Chips & Fried Chicken
Crispy golden fries with a well-seasoned piece of fried chicken
KES 200.00
Add to Cart
Spaghetti Bolognese
Al-dente spaghetti with seasoned minced beef in rich tomato sauce
KES 150.00
Add to Cart
Rice & Tilapia Fillet
Golden-fried Nile tilapia fillet served with steamed rice and salad
KES 200.00
Add to Cart
Menu

Figure 24 shows the Menu screen. The RecyclerView data originates from FirestoreRepository.listenToMenuItems(), which attaches a Firestore real-time snapshot listener. Menu changes made by staff or admin are reflected on the student's screen within seconds without any manual refresh.

The CartFragment displays all items currently held in CartViewModel, which is a shared ViewModel that survives fragment switches. Each row in the cart RecyclerView shows the item name, unit price, and a quantity stepper with minus and plus buttons. Tapping minus on an item with quantity 1 removes it from the cart entirely. A summary section below the list shows the subtotal and an estimated wait time. The estimated wait is computed by querying the count of orders currently in "pending" or "preparing" status and multiplying by a constant of five minutes per order (minimum displayed is five minutes if the queue is empty). A RadioGroup beneath the summary lets the student toggle between wallet payment and cash payment. When wallet is selected, the current balance from WalletViewModel is displayed alongside the option. If the balance is insufficient, a warning label appears and the "Place Order" button remains disabled until the student switches to cash or the balance is topped up. Tapping "Place Order" triggers the appropriate Firestore transaction based on the selected payment method.

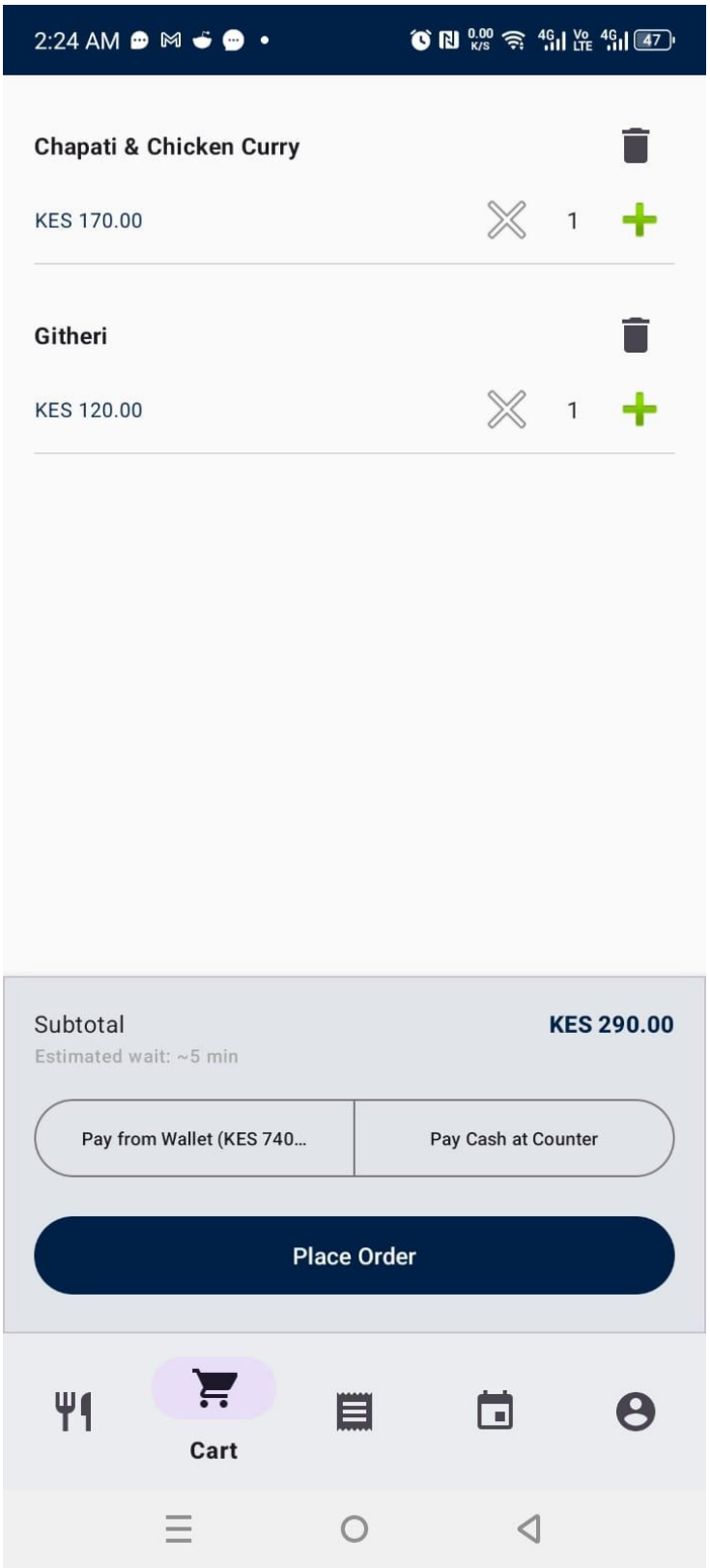

2:24 AM
Chapati & Chicken Curry
KES 170.00
1
Githeri
KES 120.00
1
Subtotal
KES 290.00
Estimated wait: ~5 min
Pay from Wallet (KES 740...
Pay Cash at Counter
Place Order
Cart

Figure 25 shows the Cart screen. The wallet balance on the payment toggle is sourced from WalletViewModel's real-time Firestore listener and updates automatically if a top-up occurs while the student is reviewing their cart. The insufficient-balance guard disables the Place Order button locally - the check runs against the LiveData balance before any Firestore write is attempted, so no server round-trip is needed to enforce it.

iii. Student Interface - Orders, Pre-orders, and Wallet

The OrdersFragment presents a two-tab layout. The "Active" tab shows orders where the status is "pending", "preparing", or "ready", each displayed as a card with a status chip coloured amber (pending), blue (preparing), or green (ready). A real-time Firestore snapshot listener attached in FirestoreRepository.listenToStudentOrders() ensures that when cafeteria staff advance an order's status, the chip on the student's screen updates without requiring a manual refresh. When an order reaches "ready", the student also receives a Firebase Cloud Messaging push notification. The "History" tab displays orders with "collected" status in reverse chronological order.

The PreOrdersFragment shows the student's scheduled pre-orders as a scrollable list. A floating action button opens a bottom sheet dialog where the student selects a meal slot (Lunch or Dinner), a scheduled date, and items from the menu. A recurring toggle presents day-of-week checkboxes for weekly repetition. Validation ensures the selected date is not in the past and that, if today is chosen, the current time is before the cut-off for the selected slot (10:00 AM for Lunch, 5:00 PM for Dinner). Students may cancel a pre-order only while its status is "scheduled", before the cut-off processing has occurred. No wallet deduction takes place at scheduling time; deduction occurs at the cut-off via the Supabase Edge Function. The ProfileWalletFragment displays the student's current wallet balance in a prominent card at the top of the screen, with an informational note indicating that top-ups are handled at the cafeteria counter. Below the balance card, a RecyclerView backed by a WalletTransactionAdapter lists all wallet transactions in reverse chronological order. Each row shows a directional icon (green arrow for credits, red arrow for debits), the transaction description (e.g, "Top-up by staff" or "Order #abc123"), the amount, and the timestamp. A logout button at the bottom calls FirebaseAuth.signOut() and starts LoginActivity with the back stack cleared.

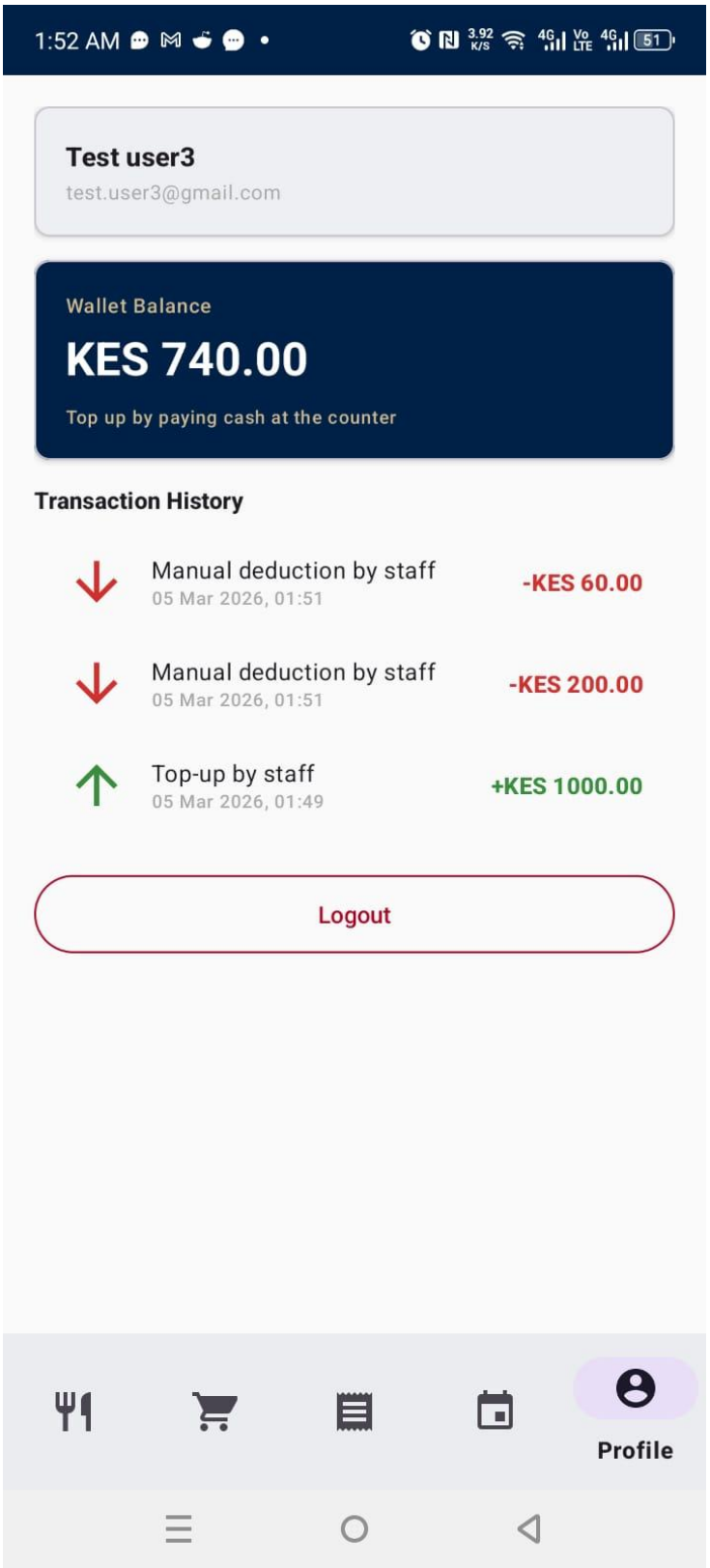
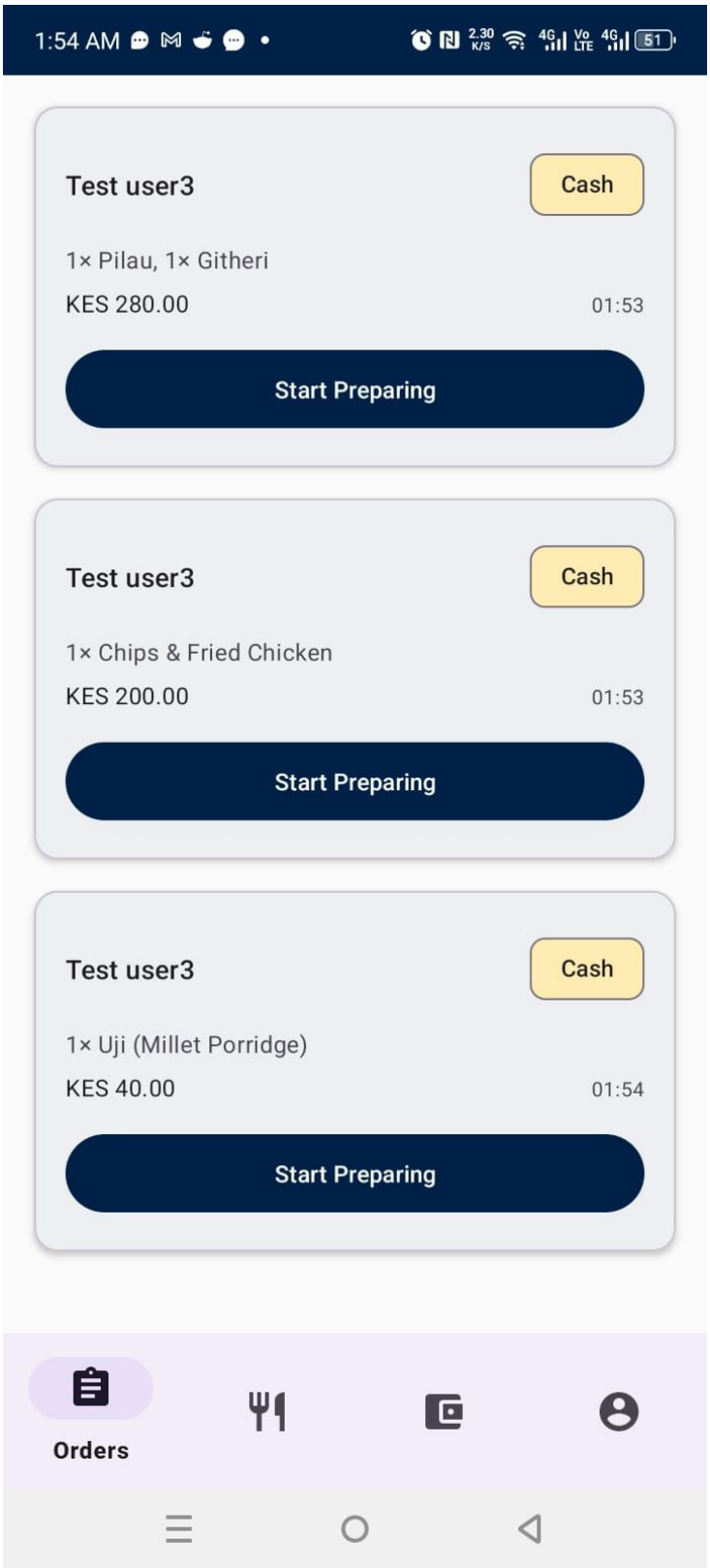

1:52 AM
Test user3
test.user3@gmail.com
Wallet Balance
KES 740.00
Top up by paying cash at the counter
Transaction History
Manual deduction by staff
05 Mar 2026, 01:51
-KES 60.00
Manual deduction by staff
05 Mar 2026, 01:51
-KES 200.00
Top-up by staff
05 Mar 2026, 01:49
+KES 1000.00
Logout
Profile

Figure 26 shows the Profile/Wallet screen. The balance card sources its value from the same WalletViewModel instance shared across the app, so it reflects any top-up or deduction that occurred during the session without requiring the student to navigate away and return. Each transaction row shows a directional icon, description, amount, and timestamp, giving the student a full audit trail of their wallet activity.

iv. Staff Interface

The staff interface is hosted in a separate StaffMainActivity with its own BottomNavigationView presenting three tabs: Orders, Menu, and Wallet. This separation ensures that staff users never see student navigation items and vice versa, reducing cognitive load and preventing accidental interactions. The StaffOrdersFragment is the primary operational screen for cafeteria staff. It displays all active orders across all students - those in "pending", "preparing", or "ready" status - in a single real-time RecyclerView. Each card shows the student name, a list of ordered items with quantities, the total amount, the payment method (wallet or cash), and the current status. An action button on each card advances the order to its next state: "Start Preparing" moves a pending order to preparing; "Mark Ready" moves a preparing order to ready and simultaneously triggers an HTTP POST to the notify-order-ready Supabase Edge Function, which sends a push notification to the student's device. "Mark Collected" finalises the order. For cash orders, the "Mark Collected" step reminds the staff member to collect physical payment before confirming.


1:54 AM
Test user3
Cash
1× Pilau, 1× Githeri
KES 280.00
01:53
Start Preparing
Test user3
Cash
1× Chips & Fried Chicken
KES 200.00
01:53
Start Preparing
Test user3
Cash
1× Uji (Millet Porridge)
KES 40.00
01:54
Start Preparing
Orders

The StaffMenuFragment provides a CRUD interface for menu items. Each item card includes an availability toggle (Switch) that staff can flip to immediately mark an item as unavailable without reloading the screen. A floating action button opens an "Add Item" dialog with fields for name, description, category, price, image URL, and availability. Tapping an existing item opens a pre-filled edit dialog. All changes are written to the menuItems Firestore collection and immediately reflected on student devices through their active snapshot listeners.

The StaffMenuFragment provides a CRUD interface for menu items. Each item card includes an availability toggle (Switch) that staff can flip to immediately mark an item as unavailable without reloading the screen. A floating action button opens an "Add Item" dialog with fields for name, description, category, price, image URL, and availability. Tapping an existing item opens a pre-filled edit dialog. All changes are written to the menuItems Firestore collection and immediately reflected on student devices through their active snapshot listeners.

The StaffWalletFragment allows staff to manage student wallet balances. The screen presents a lookup form where the staff member enters a student ID. Tapping "Find Student" queries the users collection and displays the student's name and current balance. The staff member then enters an amount and selects either "Credit" (top-up) or "Debit" (deduction, for cases such as correcting an overpayment). Tapping "Confirm" calls the appropriate FirestoreRepository method - creditWallet() or deductWalletByStaff() - both of which use Firestore runTransaction() for atomicity and record the staff member's UID in the staffId field of the resulting walletTransactions document for audit purposes.

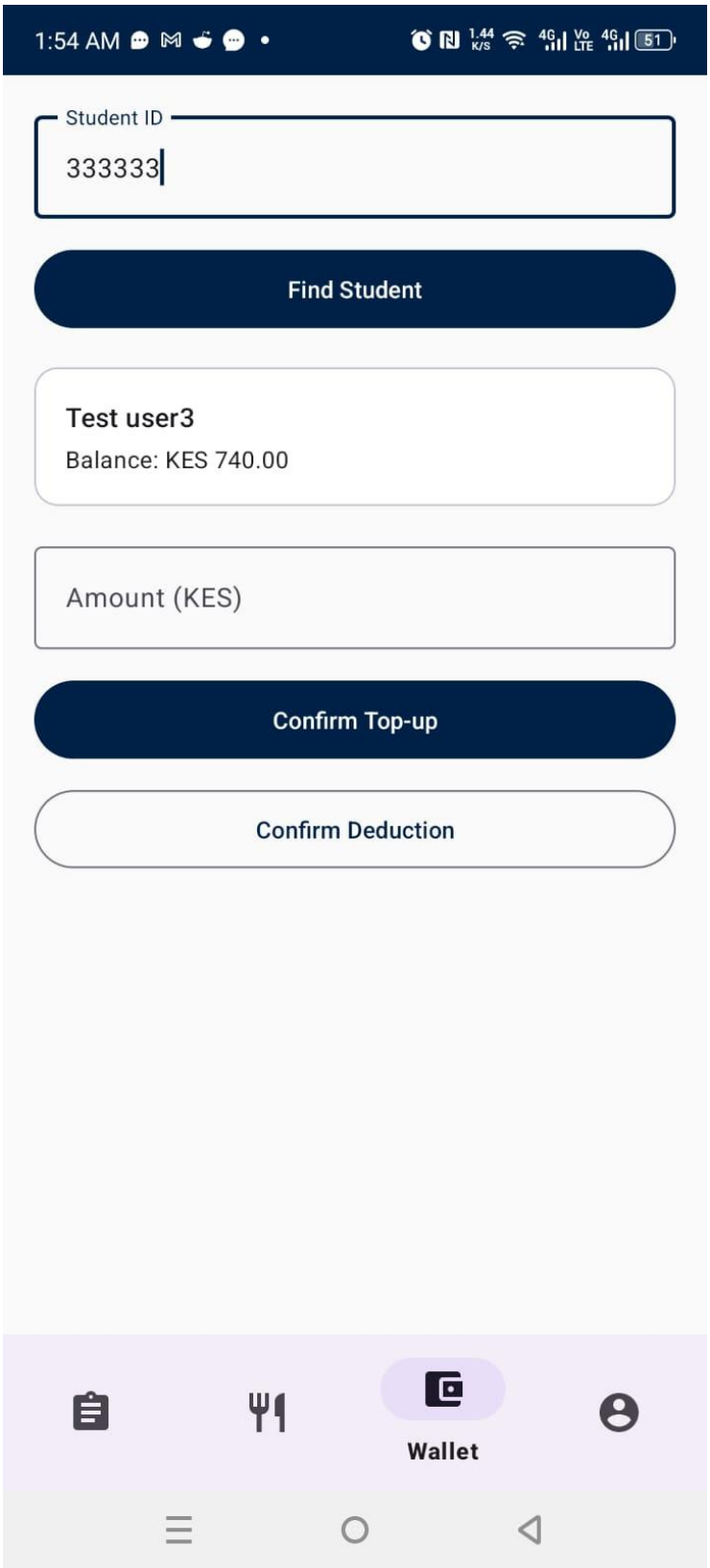

1:54 AM
Student ID
333333
Find Student
Test user3
Balance: KES 740.00
Amount (KES)
Confirm Top-up
Confirm Deduction
Wallet

Figure 28 shows the Staff Wallet screen. The student ID lookup queries the users collection before any balance operation is initiated, ensuring the staff member confirms the correct student name and current balance before committing a credit or debit. Both operations use Firestore runTransaction for atomicity and embed the staff member's UID in the resulting walletTransactions document, providing a full audit trail.

v. Admin Web Panel

The web-based admin panel is a separate Next.js 14 application deployed independently of the Android app. It is accessible only via desktop browser and is role-gated: login is handled through the Firebase Client SDK, and upon authentication the panel reads the user's role field; any non-admin user is redirected to the login page. Admin accounts with firstLogin: true are redirected to a change-password page before accessing the dashboard. The dashboard consists of four pages. The Menu Management page presents a full-width data table of all menu items with inline availability toggles, edit buttons, and delete buttons. The Staff Account Creation page provides a form for entering the new staff member's name, email, and temporary password; submission calls a Next.js API route (/api/create-staff) which uses the Firebase Admin SDK to create the Firebase Auth user and write the Firestore document with role: "staff" and firstLogin: true. The Users Directory page displays two separate tables - one for students and one for staff and admin accounts - both read from Firestore, filtered by role, providing the administrator with a complete view of all system users.

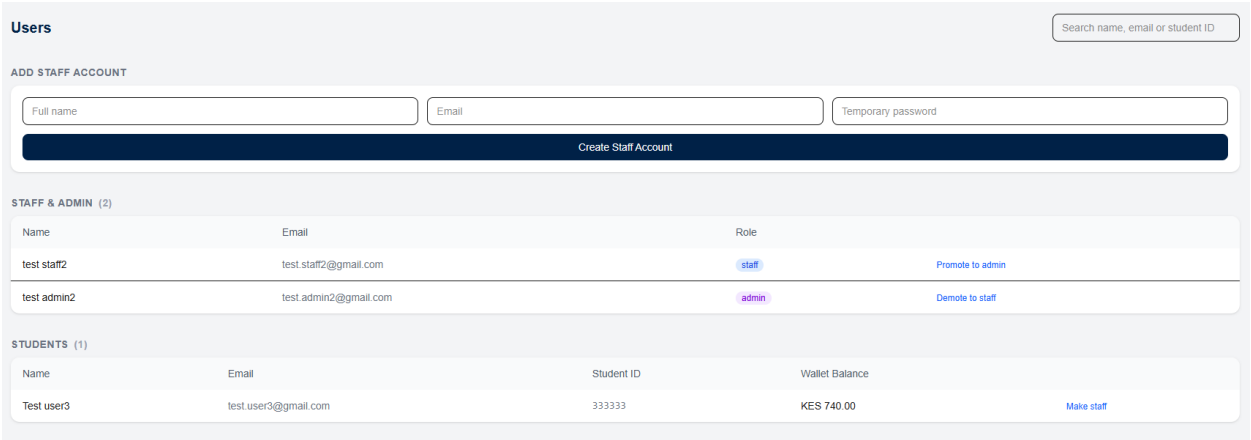


Figure 29 shows the admin panel's staff account creation interface. The Admin SDK API route runs server-side, keeping the service account credentials off the client and ensuring that only the server can create Auth users with arbitrary roles.

6.3 Backend Implementation

The backend architecture of the USIU Cafeteria Ordering System is built on Firebase's managed cloud services supplemented by Supabase Edge Functions for scheduled automation. Unlike a traditional server-client model requiring a dedicated application server, the system uses Firebase Firestore as both the real-time database and the source of truth for all application state, Firebase Authentication for identity management, and Firebase Cloud Messaging for push notifications. Supabase Edge Functions - serverless TypeScript functions running on the Deno runtime - handle the two automation tasks that require a server-side execution context: processing pre-order cut-offs on a schedule, and dispatching push notifications when an order status changes to "ready". The entire backend operates on free-tier plans with no ongoing operational cost.

i. Firestore Data Model

The system's data is organised across five Firestore collections at the root of the database. The users collection stores one document per registered user, identified by the Firebase Auth UID. Each document contains the user's name, email, studentId, role ("student", "staff", or "admin"), walletBalance (present only for students), fcmToken (the device's current FCM registration token), and firstLogin (a boolean that is true until the user completes their first-login password change). Staff and admin users do not have a walletBalance field; their documents contain only identity and role data.

The menuItems collection holds one document per menu item, with fields for name, description, price, category ("breakfast", "lunch", or "dinner"), imageUrl, and available (a boolean availability flag). The orders collection stores customer order documents, each containing an embedded list of ordered items (name, price, quantity per item), the total amount, payment method, status, and an estimatedWaitMin value calculated at order creation time. The preOrders collection holds pre-order documents with a scheduledDate field (formatted "YYYY-MM-DD"), mealSlot, recurring flag, recurringDays list, and a status field that transitions through "scheduled", "confirmed", and

"cancelled". The walletTransactions collection records every wallet credit and debit, with a staffId field on staff-initiated transactions providing an audit trail linking every balance change to the staff member who performed it.

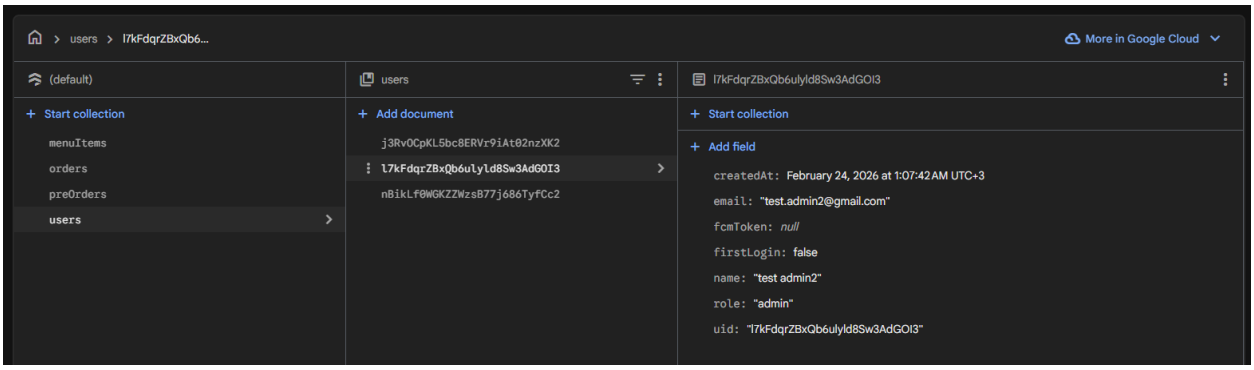


Figure 30 shows the Firestore database structure. Composite indexes are deployed for the five most performance-critical query patterns: student order history (userId ASC + createdAt DESC), staff order queue (status ASC + createdAt ASC), student pre-order listing (userId ASC + createdAt DESC), pre-order cut-off query (mealSlot ASC + scheduledDate ASC + status ASC), and wallet transaction history (userId ASC + createdAt DESC). These indexes were created via the Firebase Console and are required for Firestore to execute compound queries efficiently without throwing "index required" errors at runtime.

ii. FirestoreRepository Pattern

All Firestore reads and writes in the Android application are routed through a single class, FirestoreRepository. No Fragment or ViewModel accesses the Firestore SDK directly; they call methods on the repository, which returns LiveData objects or accepts callbacks. This pattern enforces a strict separation between UI logic and data access, makes all database operations auditable in one place, and simplifies future testing and maintenance. The repository exposes distinct methods for each data operation. listenToMenuItems() attaches a Firestore addSnapshotListener to the menuItems collection and pushes updates to a MutableLiveData list. listenToStudentOrders(userId) queries the orders collection filtered by userId, ordered by createdAt descending, and streams results via LiveData. listenToAllActiveOrders() serves the staff screen, querying for orders where status is in "pending", "preparing", or "ready". placeOrderWithWalletDeduction(order) executes a Firestore runTransaction that atomically

deducts the order total from walletBalance and writes the order and wallet transaction documents in a single server-side operation. creditWallet(userId, amount, staffId) and deductWalletByStaff(userId, amount, staffId) both use runTransaction and embed the staffId in the resulting walletTransactions document. notifyOrderReady(orderId, userId) makes an HTTP POST to the Supabase notify-order-ready Edge Function after a staff member marks an order as "ready".

```
FirestoreRepository
-db : FirebaseFirestore
-auth : FirebaseAuth
+listenToMenuItems() : LiveData<List<MenuItem>>
+listenToStudentOrders(userId : String) : LiveData<List<Order>>
+listenToAllActiveOrders() : LiveData<List<Order>>
+getActiveOrderCount() : Task<Integer>
+placeOrderWithWalletDeduction(order : Order) : Task<Void>
+placeOrderCash(order : Order) : Task<Void>
+updateOrderStatus(orderId : String, status : String) : Task<Void>
+notifyOrderReady(orderId : String, userId : String) : void
+creditWallet(userId : String, amount : Double, staffId : String) : Task<Void>
+deductWalletByStaff(userId : String, amount : Double, staffId : String) : Task<Void>
+listenToWalletBalance(userId : String) : LiveData<Double>
+loadWalletTransactions(userId : String) : LiveData<List<WalletTransaction>>
+updateItemAvailability(itemId : String, available : Boolean) : Task<Void>
+schedulePreOrder(preOrder : PreOrder) : Task<Void>
```

Figure 31: FirestoreRepository — Key Method Signatures

Figure 31 illustrates the centralised repository structure. The use of Firestore transactions in all wallet operations is a critical design decision: because Firestore transactions perform a read-then-write atomically on the server, concurrent wallet operations from multiple devices cannot result in an incorrect balance. If two operations attempt to modify the same balance simultaneously, Firestore retries the transaction automatically, ensuring consistency without any application-level locking.

iii. MVVM Architecture - ViewModels and LiveData

The Android application follows the Model-View-ViewModel (MVVM) architectural pattern. ViewModels are Android Architecture Components that survive configuration changes such as screen rotation and persist across fragment switches within the same activity. Fragments observe

LiveData objects exposed by ViewModels; when the data changes, the Fragment updates its UI automatically without polling. CartViewModel holds the in-memory cart state as a MutableLiveData list of CartItem objects. It exposes addItem(), removeItem(), updateQuantity(), clearCart(), and setPaymentMethod() methods called by CartFragment. Because CartViewModel is scoped to the activity, the cart persists when the student navigates between tabs and is only cleared when the student explicitly places an order or logs out.

WalletViewModel maintains a real-time listener on the authenticated user's walletBalance field in Firestore. It exposes a LiveData of type Double - a balance observable that CartFragment uses to display the current balance on the payment toggle and enforce the insufficient-balance guard. Whenever the balance changes in Firestore - due to a top-up by staff or a deduction for an order - WalletViewModel's LiveData emits the new value and CartFragment's UI updates immediately without requiring a screen refresh.

OrdersViewModel manages the student's active and historical orders, using FirestoreRepository.listenToStudentOrders() to receive real-time status updates. Each status change emitted by the Firestore listener causes the RecyclerView adapter in OrdersFragment to update the affected order card's chip colour and label.

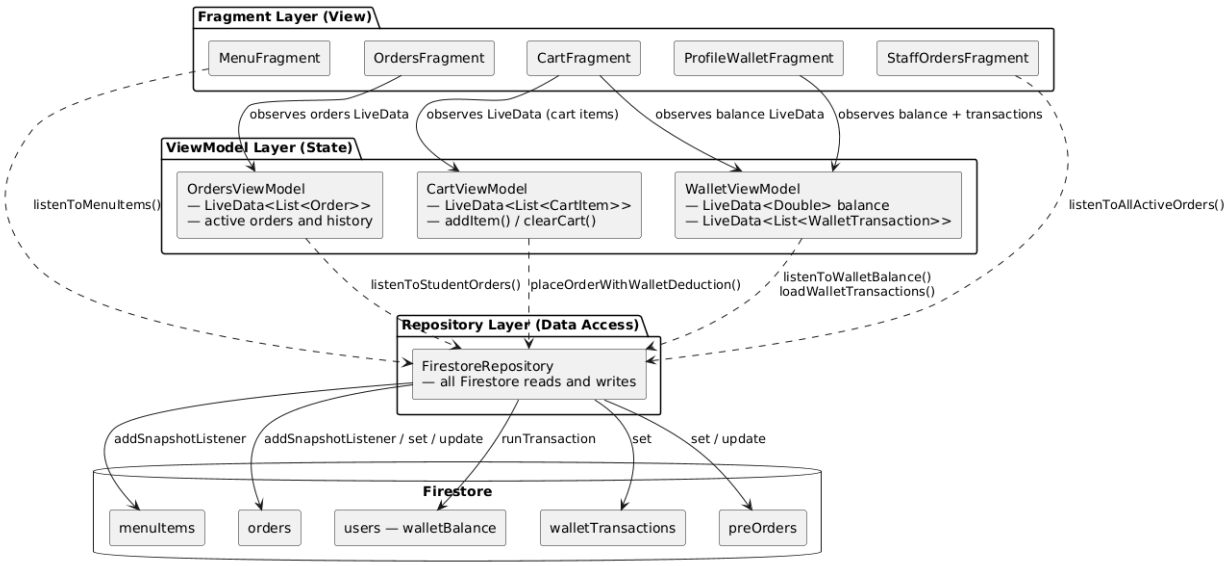


Figure 32: MVVM Data Flow — Android Application

Figure 32 illustrates the MVVM data flow. Fragments hold no direct reference to Firestore; they interact exclusively with ViewModels, which delegate all persistence operations to FirestoreRepository. This layering ensures that a screen rotation or tab switch does not restart a Firestore listener, as the ViewModel survives the Fragment lifecycle and continues emitting updates to whichever Fragment instance is currently active.

iv. Supabase Edge Functions

Two Supabase Edge Functions implement the server-side automation that requires a trusted execution environment outside the Android app. Both functions are written in TypeScript and run on the Deno runtime. They authenticate incoming HTTP requests using a shared secret (FUNCTIONS_SECRET) passed as a Bearer token in the Authorization header. The secret is stored in Supabase project settings and is also embedded in the Android app's gradle.properties file, which is excluded from version control via .gitignore to prevent its exposure in source code.

The process-cutoff function is triggered by HTTP POST requests from cron-job.org at 07:00 UTC (10:00 AM East Africa Time) for lunch and at 14:00 UTC (5:00 PM East Africa Time) for dinner. The request body specifies the mealSlot. The function uses the Firebase Admin SDK to query the preOrders collection for documents where status is "scheduled", scheduledDate matches today's date, and mealSlot matches the target slot. For each matched document, it reads the student's walletBalance inside a Firestore transaction. If the balance is sufficient, it deducts the pre-order total, sets the pre-order status to "confirmed", creates a walletTransactions document, and sends a confirmation FCM notification. If the balance is insufficient, it sets the status to "cancelled" and sends a cancellation notification. For recurring pre-orders, the function creates a new preOrders document for the next weekly occurrence with status "scheduled", maintaining the automatic renewal chain without any student action. The notify-order-ready function is called by the Android app via an HTTP POST immediately after the Firestore write that sets an order's status to "ready". The function reads the student's fcmToken from their users document and sends an FCM push notification informing the student that their order is ready for collection. This replaces the Firestore trigger approach that would require Firebase Cloud Functions on the Blaze plan; instead, the

Android app takes responsibility for triggering the notification as a best-effort background call after the Firestore status update succeeds.

A critical implementation detail for both functions is the line db.settings({ preferRest: true }) called on the Firestore Admin SDK instance immediately after initialisation. The Deno runtime does not support gRPC, which is the default transport protocol for the Firebase Admin SDK's Firestore client. Without this setting, all Firestore calls from the edge functions fail silently. Setting preferRest to true instructs the Admin SDK to use the Firestore REST API instead, which is fully compatible with the Deno HTTP stack.

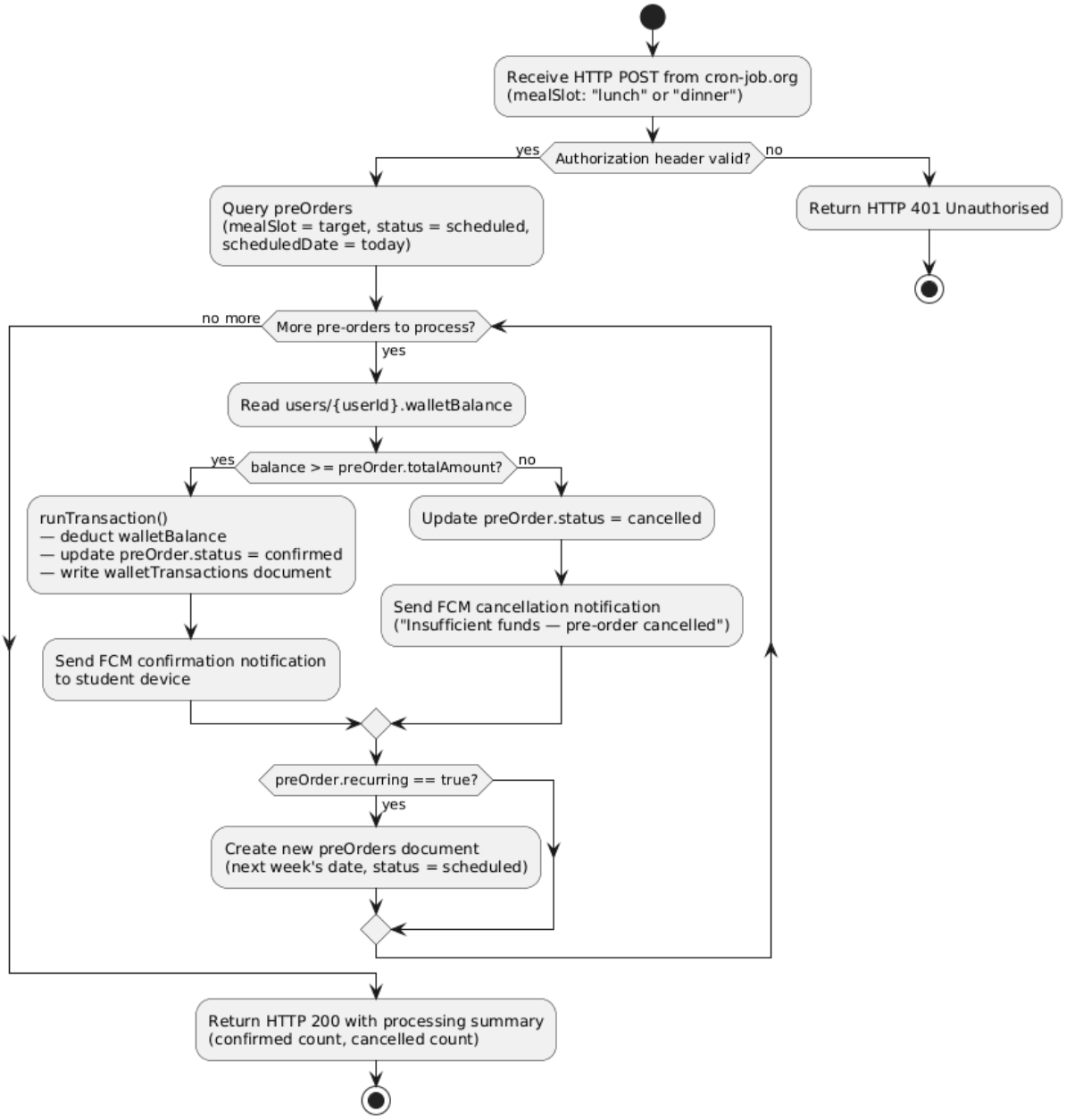


Figure 33: process-cutoff Edge Function — Logic Flow

Figure 33 shows the core logic of the process-cutoff function. The function initialises the Firebase Admin SDK with service account credentials stored in Supabase secrets, applies the preferRest setting, then executes the batch processing loop. The use of runTransaction ensures that the wallet deduction and pre-order status update are either both committed or both rolled back, preventing scenarios where a student is charged but their pre-order is not confirmed.

6.4 Integration

The integration phase of the USIU Cafeteria Ordering System aimed at establishing seamless connectivity between the Android UI layer, the FirestoreRepository data layer, the ViewModel state management layer, and the external Supabase Edge Functions. The objective was to ensure that data flows predictably through the system in all directions: menu updates from staff propagate to students in real time; cart actions in CartViewModel trigger atomic Firestore transactions; order status advances by staff notify students via push notifications; and pre-order cut-offs run automatically on a schedule without any user intervention. Through consistent architectural patterns and careful integration testing across all workflows, the system achieved a reliable, cohesive user experience.

i. Fragment Navigation Integration

The navigation backbone of the Android application is a BottomNavigationView inside each of the two main activities - MainActivity for students and StaffMainActivity for staff. Fragment switching is implemented using the hide/show pattern rather than the standard replace pattern. When the user taps a navigation item, the currently visible fragment is hidden (hide()) and the target fragment is shown (show()). If the target fragment has not yet been added to the back stack, it is created and added (add()) at that point. Subsequent visits to the same tab simply call show() on the already-created instance.

This approach has a critical practical advantage for the cafeteria context: the MenuFragment's RecyclerView scroll position and the OrdersFragment's tab selection are preserved when the student navigates away and back. If replace() were used instead, each return to the fragment would reconstruct the view hierarchy from scratch, losing state and causing an unnecessary re-attachment of Firestore listeners.

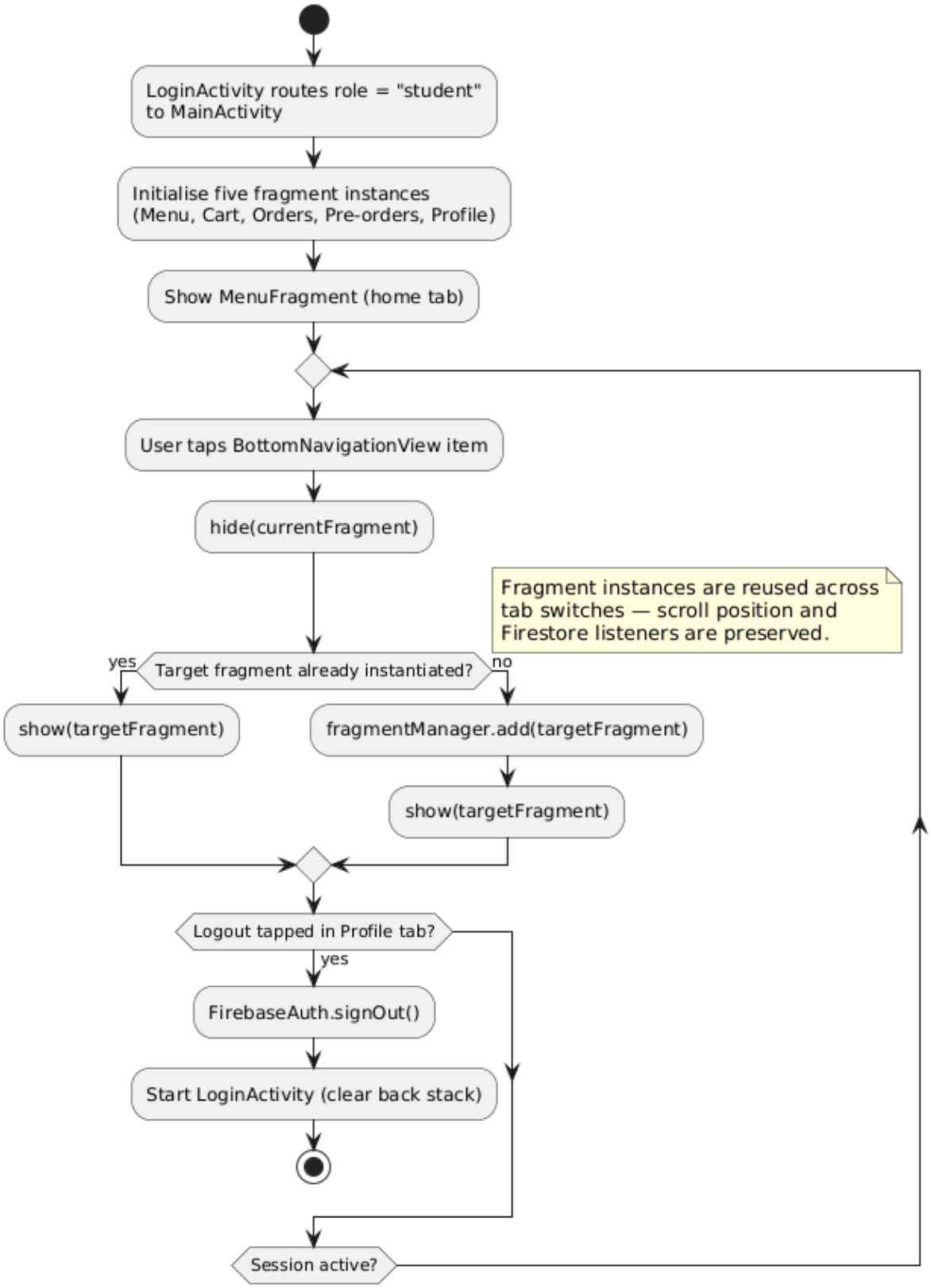


Figure 34: MainActivity — Fragment Navigation (hide/show pattern)

Figure 34 shows the fragment management logic inside MainActivity. The activity holds references to all five fragment instances. When a navigation item is selected, the transaction manager hides all other fragments and shows the selected one. This ensures only one Firestore snapshot listener per collection is active at any time within the student session, preventing duplicate data emissions.

ii. Cart and Wallet Integration

The cart workflow integrates CartViewModel, WalletViewModel, and FirestoreRepository in a coordinated sequence. The student builds the cart entirely in memory using CartViewModel - no Firestore writes occur until "Place Order" is tapped. This design keeps Firestore costs low and avoids creating incomplete order documents in the database. When "Place Order" is tapped with wallet payment selected, CartFragment first checks WalletViewModel's current balance against the cart subtotal. If insufficient, it shows a dialog and does not proceed. If sufficient, it calls FirestoreRepository.placeOrderWithWalletDeduction(), which executes a Firestore runTransaction containing three operations: reading the current walletBalance, deducting the order total, and writing both the new order document and the walletTransactions document. Because all three operations run atomically inside the transaction, the system guarantees that either all succeed together or none are committed. This prevents partial states such as a deduction without an order record, or an order created without a corresponding balance reduction.

For cash orders, CartFragment calls FirestoreRepository.placeOrderCash(), which writes only the order document with paymentMethod set to "cash". No wallet check or deduction occurs. On success in both paths, CartViewModel.clearCart() is called, the student is navigated to the Orders tab, and a snackbar confirms the placement. The estimated wait time displayed in CartFragment is computed once when the fragment resumes by calling FirestoreRepository.getActiveOrderCount(), which runs a one-shot Firestore query counting documents with status "pending" or "preparing". The result is multiplied by five (minutes per order) and displayed as "Estimated wait: ~X min". A minimum of five minutes is shown even when the queue is empty, representing the preparation time for the student's own order.